

ATHL Fan Token Platform Requirements

[ATHL Fan Token Platform Requirements](#)

[Issuer Sign Up](#)

[Athlete-Selected Button - Page One](#)

[Creator Pathway](#)

[All Issuers](#)

[Integration with Identity Verification](#)

[Bio Development and Minting Preferences](#)

[Alchemy Wallet Creation \(ALL USERS\)](#)

[Sign-In Process](#)

[Login Method Selection Based on User Type](#)

[Referral Engine \(*This section Will be edited after Tom & Paul finalize new referral program\)](#)

[Waiting List Creation and Management](#)

[Fan Dashboard](#)

[Issuer Specific Purchase Page](#)

[Issuer Admin Pages](#)

[Developing Fantasy Teams](#)

[Team Competitions](#)

[Fantasy Betting System](#)

[Team Leaderboard](#)

[ATHLScan Database For Public Viewing](#)

[ATHL Customer Support Request Page](#)

[Requirements to Foster an Active Trading Environment](#)

[Security Requirements for ATHL Follower Token Trading Platform](#)

[Purchase Anomaly Detection and CAPTCHA Verification](#)

[Randomized Purchase Execution Delays](#)

[User Risk Score Calculation](#)

[User Risk Score Triggers](#)

[Non-X Social Media Monitoring](#)

[Grok-Based X Social Media Monitoring](#)

[Strong Email Verification for User Accounts](#)

[Multi-Factor Authentication \(MFA\) with Flexible Options](#)

[Behavioral Biometrics for Bot Detection](#)

[User Risk Score Transparency](#)

[Monitor Trading Patterns for Manipulation with Staff Alerts](#)

[Dynamic Trading Pause Triggers](#)

[Real-Time Processing](#)

[Scalability](#)

[Compliance](#)

[User Experience](#)

Adaptability

Updated ATHL Security Requirements: Polymarket-Related Additions

Integration with Existing Requirements

Additional Requirements To Do

Issuer Sign Up

Access and Flow

- **FR-1:** The Issuer Sign Up process shall be accessible via a dedicated page (e.g., <https://athl.live>) linked from the platform's introductory page.
- **FR-4:** To get to the sign up forms, users must click either "Athlete" or "Creator", or "Fan".

DATA POINTS:

Issuer table FR-4?

Field	Description	Type	Required	Notes
issuer_id	Unique internal user ID	UUID	Yes	System-generated
issuer_type	Athlete or Creator	Enum	Yes	Controlled via UI toggle FR-4
identity_verified	If issuers identity is verified	Boolean	Yes	FR-21
display_name	What should we call you?	Text	Yes	
email	Primary contact email	Email	Yes	Validated + verified
gender	Gender	Enum	Yes	Male, Female, Other, Prefer not to say
country	Country of residence	Enum	Yes	ISO alpha-2 code
zip_code	Postal/ZIP code	Text	Yes	Format-validated
sign_up_timestamp	Timestamp of registration	Timestamp	Yes	System-generated
status	Competitive, Recreational, Retired	Enum	Yes	Self-selected

Athlete-Selected Button - Page One

- **FR-5:** The sign-up form shall collect the following mandatory personal information:
 - {By order of entry}

- What should we call you? {Txt, *, single field}
- Email: {Txt, REQ, single field, ensure valid email format}
- See the sections Strong Email verification and Multi-Factor Authorization in the Security Section.
- Primary social platform: (Drop down menu, REQ: Instagram, Twitter(X), TikTok, LinkedIn, GitHub, Discord, Spotify, Instagram, Telegram, Farcaster, Facebook, YouTube, Pinterest, Reddit, SoundCloud, Vimeo, OnlyFans, Shopify)
 - We will need to use OAuth v 2.0 to verify All of these socials except OnlyFans can be verified with OAuth.
- Handle for primary social
 - Handle API call... key is to instantly database every piece of data pulled to limit need for API calling and dependencies.
 - Create internal search model (elastic search?)
 - Autocalc key data on followers for flagging and recs of token supply
- Secondary social platform: (Drop down menu, REQ: Instagram, Twitter(X), TikTok, LinkedIn, GitHub, Discord, Spotify, Instagram, Telegram, Farcaster, Facebook, YouTube, Pinterest, Reddit, SoundCloud, Vimeo, OnlyFans, Shopify, other)
 - If “Other” is chosen, provide a Text box, free entry
- Handle for secondary social(Txt, Req)

Athlete-Selected Button - Page Two

- **FR-12:** Using dropdown menus unless specified (SEE ATHL-100 for menu contents):
 - **Country** (REQ, dropdown menu, standard country list).
 - ADD to Figma
 - **Zip code** (REQ, text input, validated for format based on country).
 - Add to Figma
 - **Gender** (REQ dropdown menu, e.g., Male, Female, Other/Prefer not to say).
 - Add to Figma
 - **Sport:** (REQ, Dropdown Menu see **ATHL-100 “Position by Sport”** page, if their sport is not in the list, have “Other” as the last option, then allow them to add it via Text box)
 - **Position:**(REQ, Dropdown Menu see ATHL-100 “Position by Sport” page, if their position is not in the list, have “Other” as the last option, then allow them to add it via Text box, free from))

- The system shall dynamically show/hide athlete-specific fields based on the athlete type selection, ensuring only relevant fields are displayed. (ie. if football is selected it should only show football positions).
 - Athletes can choose multiple positions for each sport they play.
- Athletes should be able to select secondary and tertiary sports and their respective positions. (Optional: repeating sports and position steps with drop down menus)
 - **ADD to Figma**
- **Level:** (REQ, Dropdown Menu: Youth/Grade School, High School, College, Amateur, Semi-Pro, Professional, Recreational)
- **League:**
 - College: Drop down menu: If the athlete, please see the **ATHL-100 “NCAA + Others”** page for lists of leagues, with all college names by state. (REQ, have an “other” option for people to write in league, Text box, free input)
 - IF Pro and USA: See additional pages in **ATHL-100 “Pro Leagues & Teams”** page to create drop down menus. (REQ, have an “other” option for people to write in league, Text box, free input)
 - The system shall dynamically show/hide athlete-specific fields based on the athlete type selection, ensuring only relevant fields are displayed. (ie. if football is selected it should only show football leagues).
 - All others: (REQ: Text Box, free input)
- **School Name or Team Name:** .
 - If USA athlete in college, (REQ, Drop down menu: Please see the **ATHL-100 “NCAA + Others”** page for lists of leagues, with all college names by state, have an “other” option for people to write in school name, Text box, free input)
 - IF Pro and USA: (REQ, Drop down menu: See additional pages in **ATHL-100 “Pro Leagues & Teams”** page to create drop down menus. Drop down menus are only for select sports. Have an “other” option for people to write in Team name, Text box, free input)
 - All others: (REQ; Text box, free input)
 - **School hash tag!!**
 - **Need option for “other”**

- **Status:** (REQ, Dropdown Menu: Competitive (league refereed games), Recreational, Retired)
 - **Add to Figma**

DATA POINTS:

athlete_profile table FR-12

Field	Description	Type	Required	Notes
issuer_id	Issuer foreign key	UUID	Yes	
sport_name	List of sports	Enum	Yes	ATHL-100 for menu contents
sport_id	Sport ID	ID	Yes	Foreign key to sports table
ordinal	Primary/Secondary/Tertiary	Enum	Yes	
positions	List of positions	Array<Text>	Yes	Multiple allowed
position_id	Position ID	ID	Yes	Foreign key to positions table
level	Competition level	Enum	Yes	
league	League or governing body	Text	Yes	From dropdown or freeform
team_or_school	Team or school name	Text	Yes	From dropdown or freeform
school_hashtag	Associated hashtag	Text	No	

Creator Pathway

- **FR-14:** If the user selects “Creator” as their creator type, the form shall display the following fields, using dropdown menus (per **ATHL-100 “Creator Types + Music”** tab for menu contents):
 - **Creator Type:** (Users can select multiple options. REQ, Drop Down menu, Have an “other” option for people to write in Team name, Text box, free input)

- **ONLY If Musician: Music Type:** (Users can select multiple options, REQ ONLY IF MUSICIAN, Drop Down menu, Have an “other” option for people to write in music type,Text box, free input)
- **Primary Content Delivery Medium:** E.g., (Users can select multiple options. REQ, Drop Down menu, Have an “other” option for people to write in Team name,Text box, free input)

DATA POINTS:

creator_profile table FR-14

Field	Description	Type	Required	Notes
issuer_id	Issuer foreign key	UUID	Yes	
creator_types	Content roles	Array<Enum>	Yes	
music_types	Only if Musician selected	Array<Enum>	Conditional	
delivery_mediums	Primary content channels	Array<Enum>	Yes	e.g., YouTube, Blog

All Issuers

Social Media Verification and Follower Count Capture - All Issuers

- **FR-15: Social Media Platform Selection and Verification Methods**
 - The system shall use the social media platform and handle collected in previous steps. Available platforms include: Instagram, Twitter(X), TikTok, LinkedIn, GitHub, Discord, Spotify, Telegram, Farcaster, Facebook, YouTube, Pinterest, Reddit, SoundCloud, Vimeo, OnlyFans, Shopify, other.

DATA POINTS:

issuer_social_accounts table FR-15 and FR-17

Field	Description	Type	Required	Notes
id	Record ID	UUID	Yes	Primary Key
issuer_id	Issuer foreign key	UUID	Yes	Foreign Key
platform	e.g., Instagram, TikTok	Enum	Yes	
platform_id	Platform foreign key	ID	Yes	Foreign Key
ordinal	Primary/Secondary/Tertiary	Enum	Yes	
handle	Username on platform	Text	Yes	
handle_type	Primary or Secondary	Enum	Yes	

verified	Whether verified	Boolean	Yes	
verification_method	OAuth, Manual, Challenge	Enum	Yes	OAuth, Auth0, Challenge, Manual
follower_count	Follower count	Integer	Yes	
above_300k	If the follower count is above 300K (high profile issuer)	Boolean	Yes	
last_verified	Last verified timestamp	Timestamp	Yes	
metadata	Additional platform data	JSON	No	Optional

- Verification methods shall include:
 - **OAuth or AuthO-Based Verification:**
 - Use Privy for OAuth on supported platforms (Instagram, X, TikTok, LinkedIn, GitHub, Discord, Spotify, Telegram, Farcaster, Shopify).
 - Use OAuth 2.0 for platforms not supported by Privy (Facebook, YouTube, Pinterest, Reddit, SoundCloud, Vimeo), prioritizing YouTube Data API for YouTube and Instagram Graph API for Instagram.
 - IF we go with Alchemy for wallets, then Use Auth0 ensuring coverage for Facebook and YouTube. We may need to use OAuth 2.0 for some socials not covered by AuthO.
 - **Challenge-Based Verification (Fallback):**
 - For platforms without reliable APIs or failed OAuth generate a unique code (e.g., "ATHL-X7B9Q2") during sign-up and instruct the issuer to post it in a platform-specific location
 - **Instagram:** Bio (profile description)
 - **Twitter(X):** Bio (profile description)
 - **TikTok:** Bio (profile description)
 - **LinkedIn:** About section (profile summary)
 - **GitHub:** Profile README (public repository or user profile description)
 - **Discord:** User profile status or About Me section
 - **Spotify:** Artist profile bio (via Spotify for Artists)
 - **Telegram:** Channel or group description (if issuer manages a public channel/group)
 - **Farcaster:** Profile bio
 - **Facebook:** Page bio (for public pages) or About section
 - **YouTube:** Channel description (About tab)

- **Pinterest:** Profile description
 - **Reddit:** User profile bio (About section)
 - **SoundCloud:** Profile bio
 - **Vimeo:** Profile bio
 - **OnlyFans:** Bio (profile description)
 - **Shopify:** Store bio (About page or footer description)
- The system shall check for the code via API (if available) or manual review within 24 hours. For API-supported platforms, poll every 5 seconds for up to 1 hour after code delivery.
- **OnlyFans-Specific Verification:**
 - Generate a unique code (e.g., "ATHL-X7B9Q2") and instruct creators to add it to their OnlyFans bio.
 - Subscribe to onlyfansapi.com (\$99/month), connect via /authenticate, and use /users/{username} to fetch bio data. Poll every 5 seconds to detect the code.
 - Fallback: If bio check fails, request the code in a post and check via /posts (only if we do not have to subscribe to creator). Alternatively, possibly accept a screenshot of the verified dashboard for manual confirmation.
 - Safety: Use a virtual card for payments to onlyfansapi.com and monitor OnlyFans' terms to avoid bans.
- **FR-16: Social Media Account Ownership Verification**
 - The system shall use OAuth 2.0, Auth0, or platform-specific APIs (e.g., YouTube Data API, Instagram Graph API, X API) to verify the user's primary and secondary social media accounts and capture their follower counts.
 - Verification shall confirm that the user controls the provided social media handle/channel (e.g., EpicGamingWizard) 2, using:
 - OAuth 2.0 or Auth0 for platforms supported by Privy, Alchemy, or direct APIs (per FR-15).
 - Challenge-based or manual verification (per FR-15) only for platforms without reliable APIs or if OAuth/Auth0 fails.
 - OAuth 2.0 shall verify professional Instagram accounts through the Graph API and non-professional accounts through the Basic Display API.
- **FR-17: Social Media Verification Process**
 - The system shall:
 - Prompt the user to authenticate their social media accounts during sign-up using OAuth, AuthO, or challenge-based.
 - Validate that the provided handles match the authenticated accounts.
 - Store the verification outcome (e.g., "(social platform name) Login Verified" with a checkmark or "(Social platform name) Login Pending") for display on the Issuer Specific Purchase Page (aligned with FR-10).

- Store the verified follower counts with timestamps in the database to track follower growth velocity.
- Capture as much relevant metadata as you can (e.g., account email, phone, secondary email and phone, creation date, IP, Location, linked socials, etc) where available to support alias verification.
- At least two social media accounts from the dropdown list must be verified.
- **FR-19: Encouraging Additional Social Media Verifications**
 - The system shall encourage issuers to verify all social media accounts with significant followers (up to five additional accounts).
 - Repeat verification steps (per FR-6 and FR-17) for each additional platform:
 - Additional social platform: (Dropdown, OPTIONAL: Instagram, X, TikTok, LinkedIn, GitHub, Discord, Spotify, Telegram, Farcaster, Facebook, YouTube, Pinterest, Reddit, SoundCloud, Vimeo, OnlyFans, Shopify, other).
 - Handle for additional platform: (Text, Required if platform selected).
 - Store verification outcomes and follower counts as described in FR-17.

DATA POINTS:

social_verification table FR-16 and FR-17

Field	Description	Type	Required	Notes
verification_id	Verification record ID	UUID	Yes	Primary key
issuer_id	Issuer foreign key	UUID	Yes	Links to issuers table
platform	Social platform	Enum	Yes	e.g., Instagram, X, TikTok
platform_id	Platform foreign key	ID	Yes	Foreign Key
handle	Social handle	Text	Yes	
verification_method	OAuth, Auth0, Challenge, Manual	Enum	Yes	
challenge_id	Challenge code id	UUID	No	
challenge_code_location	Expected location for challenge code	Text	No	e.g., bio, description
verification_status	Verified, Pending, Failed	Enum	Yes	
verified_timestamp	Timestamp of last verification	Timestamp	Yes	

metadata	Additional verification metadata	JSON	No	Includes alias indicators, match scores, etc.
----------	----------------------------------	------	----	---

- **R-18: Error Handling for Social Media Verification**

- The system shall display an error if social media authentication fails (e.g., “Unable to verify YouTube channel”) and allow the user to retry up to three times per platform.
- If verification fails after retries, flag the account as “Ownership Verification Failed” and provide a support link for manual verification (per FR-6).

DATA POINTS:

social_media_auth_fail table R-18

Field	Description	Type	Required	Notes
issuer_id	Issuer foreign key	UUID	Yes	
mfa_enabled	Multi-factor auth active	Boolean	Yes	
sign_up_ip	IP address at signup	IP	Yes	
user_agent	Device/browser info	Text	Yes	For fraud analysis
retry_attempts	Failed verification tries	Integer	Yes	Per platform basis
attempt_id	Attempt ID	UUID	Yes	Primary Key
failed_verification_flag	Ownership verification failed	Boolean	Yes	
platform_id	Platform ID Foreign Key	Array<Text>	Yes	Foreign Key
attempt_time_stamp	Datetime attempted	Timestamp	Yes	
terms_accepted	Versioned ToS agreement	Text	Yes	E.g., tos_v1.1

Integration with Identity Verification

Automatic Identity Verification (High Profile Issuers Only)

- **FR-20:** The system shall automatically initiate identity verification via Socure or a similar third-party service for issuers who meet either of the following criteria:
 - Member of a professional sports league (based on “Professional” level determining during sign up.

- 300,000 or more followers on their primary or secondary social media accounts (X, Instagram, or YouTube).
- **FR-21: Identity Verification Process**
 - For all athletes and creators using their real name, or associating their real name to their creator name/identity, The identity verification process shall:
 - Ask the issuer if they are willing to do an ID verification check (REQ, Multiple choice, Yes, No, Not right now)
 - Next to the options of “No” and “Not Right Now”, State “The label “Identification Check Not Completed” will appear in the Profile page until ID is verified”
 - If No or Not right Now is checked,
 - Add a challenge-based verification of asking them to add a special code to their primary social account to get another layer of assurance that they fully control the account.
 - If No or Not right now, add this to the User DB. We will want to send the user an email within 1 week asking again to verify ID.
 - Once challenge based verification is complete, the rest of this section can be skipped.
 - Prompt the user to provide necessary identification details (e.g., government ID, per Socure requirements - Use a Socure (or other provider link) to run this process.).
 - Display both identity verification status (e.g., Pending, Verified, Failed) and social media ownership verification status (per FR-17) during sign-up.
 - Store the identity verification outcome (checkmark for verified) and ownership verification outcome for display on the Issuer Specific Purchase Page.
 - For issuers with a mismatch between their legal name (from Socure) and verified social media handle/channel name (John Smith name for “HackerWizard” creator name), the system shall:
 - Compare the legal name from Socure with the handle/channel name provided in FR-12.
 - If a mismatch is detected (John Smith name for “HackerWizard” creator name, score the likelihood of alias ownership by matching OAuth metadata (FR-16) with Socure PII to possibly match email, phone, IP address, etc, .
 - High-confidence matches are auto-verified; low-confidence matches prompt submission of a platform-generated verification

token (FR-16) or challenge-based verification (FR-15). Manual review is a last resort for unresolved cases.”

- Display the account as “Identity Verified, Alias Review Pending” until manual review is complete.
- Manual review shall not block sign-up but shall restrict token issuance until both identity and alias reviews are complete.
- **FR-22:** The system shall allow users to complete sign-up even if identity verification is pending, but flag their account as “Verification Pending” until completed.

DATA POINTS:

Identity_verification table FR-20 and FR-21

Field	Description	Type	Required	Notes
identity_id	Identity verification ID	UUID	Yes	Primary key
issuer_id	Issuer	UUID	Yes	foreign key
user_id	Issuer user_id	UUID	Yes	foreign key
socure_id	Score evaluation ID	UUID	No	foreign key
socure_score	Socure score from Socure API	Numeric	No	
OAuth_record_id	Links to OAuth table	UUID	Yes	foreign key
challenge_id	challenge id	UUID	No	foreign key
identity_check_opt_in	User consent: Yes, No, Not now	Enum	Yes	
identity_check_status	Pending, Verified, Failed	Enum	Yes	
alias_check_status	Pending, Verified, Failed	Enum	No	
alias_match_confidence	High, Medium, Low	Enum	No	Based on cross-matching metadata
auto_verified	For high profile issuer	Boolean	Yes	Socure
Verified_ID_Alias_Review	For verified person, not verified alias	Boolean	No	Needs a better data type
verification_source	e.g., Socure	Text	Yes	
verification_result_timestamp	When result received	Timestamp	Yes	

All issuers and Fans

- **FR-7:** The form shall request a unique **ATHL username** (REQ, text input, free form), which must:
 - Be alphanumeric, 3–20 characters.
 - Be unique across all users in the platform.
 - Be validated in real-time, showing availability status (e.g., green checkmark for available).
 - Include a note that encourages the user to choose a username that is the same as one of their two largest social accounts. Autofill the users primary social handle (if available, if not then their secondary social handles) to facilitate this.
- See Security Section for : Multi-Factor Authentication (MFA) with Flexible Options

Bio Development and Minting Preferences

- The form shall collect a **bio statement** (REQ: text box, free form).
- The form shall collect **perks offered to token holders** (Optional: text box, free form).
- The form shall collect **links to personal videos or recent social media post clips** (Optional, text input for URLs and descriptions):
 - Links must be clickable, opening in a new tab or embedded player on the Issuer Specific Purchase Page.
 - Allow users to add a heading or brief descriptions of what the link is. This could all be done in the same text box, with a list of descriptions and links.
- The form shall ask the issuer how much money they would like to raise from their initial sale of their tokens. This can range from \$100,000 to \$10,000,000. This is the amount the issuer would receive, so the actual total funds raised from initial sales would be 25% higher. For instance, a 1 million raise for the issuer would be a 1.25 million raise in total.
 - Present a drop down menu that starts at \$100K and goes up by \$100K to \$1 million. Then the increment is \$1 million going up to a maximum of \$10 million.

DATA POINTS:

issuer_preferences table

Field	Description	Type	Required	Notes
issuer_id	Issuer foreign key	UUID	Yes	Foreign key

user_id	User foreign key	UUID	Yes	Foreign key
username	Unique ATHL username	Text	Yes	3-20 alphanumeric chars
username_autofill_source	Autofilled from social account	Text	No	Primary or secondary social
bio	Bio statement	Text	Yes	
perks_description	Perks offered to token holders	Text	No	
media_links	Media links and captions	JSON	No	List of links + descriptions
initial_raise_target	Funding target for token mint	Integer	Yes	In USD, actual raise = 125% of this
funding_id				

Alchemy Wallet Creation (ALL USERS)

- Upon successful submission of the sign-up form, the system shall automatically create an Alchemy wallet for the issuer.
- The system shall send an email to the issuer with:
 - Confirmation of successful sign-up and waiting list status if issuer.
 - Instructions to load the Privy wallet with USDC, USDT, ETH, or SOL.
 - A link to the wallet management interface for funding.
- The Alchemy wallet address shall be linked to the issuer's account, enabling them to purchase tokens as a fan
 - The user should then be taken to the Fan Dashboard page

Waiting List Integration

- Upon successful sign-up, the system shall automatically add the issuer to the waiting list for token launching, as per the Waiting List Creation and Management FRs (see Waiting List).

DATA POINTS:

post_signup table

Field	Description	Type	Required	Notes
issuer_id	Issuer foreign key	UUID	Yes	Links to issuer account
alchemy_wallet_id	Connects to alchemy table(s)	UUID	Yes	Created post-signup - foreign key

wallet_created_timestamp	When wallet was provisioned	Timestamp	Yes	System-generated
wallet_email_sent	Whether welcome/funding email sent	Boolean	Yes	Confirmation + instructions
wallet_ui_link	Link to wallet management interface	Text	Yes	Email + Dashboard
fan_dashboard_redirect	Did user reach fan dashboard?	Boolean	Yes	Populated from Events table
event_id	Event ID Foreign Key	UUID	No	Foreign Key
waiting_list_id	Connects to waiting list table	UUID	Yes	Foreign key to Waitlist_table

Sign-In Process

Username Input

1. The system shall require the user to enter a username in a text field (REQ, Open text).
2. The username field shall support alphanumeric characters, underscores, and hyphens, with a minimum length of 4 characters and a maximum of 20 characters (REQ, Open text)
3. The system shall validate the username exists in the database before proceeding to the next step (REQ).
4. If the username is invalid or not found, the system shall display a clear error message (e.g., "Username not found. Please try again or create an account.") (REQ).

Login Method Selection Based on User Type

5. The system shall determine the user type (Issuer or Fan) based on the provided username (REQ).
6. **For Issuers:**
 - a. The system shall require a social login using a social media platform associated with their largest or second-largest follower base, as verified during account registration (REQ).
 - b. The system shall validate the social login credentials and ensure the account matches the registered follower base (REQ).

- c. If the social login fails or the account does not match, the system shall display an error message (e.g., "Social login failed. Please use the account associated with your largest or second-largest follower base.") (REQ).
- 7. For Fans:**
- a. The system shall allow login via one of the following methods: social login, email/password, or passkey (REQ).
 - b. Supported social login platforms for Fans shall include [Twitter/X, Google, Facebook] (REQ).
 - c. For email/password login, the system shall require a valid email address and a password (minimum 8 characters, including at least one uppercase letter, one lowercase letter, one number, and one special character) (REQ)."
 - d. For passkey login, the system shall support WebAuthn-compatible passkeys (e.g., biometric authentication, hardware keys) (REQ).
 - e. If the login attempt fails, the system shall display a specific error message (e.g., "Invalid email/password" or "Passkey authentication failed") (REQ-LOGIN-14)

Multi-Factor Authentication (MFA)

MFA for Issuers

1. The system shall require MFA using an authenticator app (e.g., Google Authenticator, Authy) for all issuer accounts to enhance security, but encourage, not mandate, MFA for fan accounts.
2. The system shall guide users (issuers and fans opting in) through MFA setup upon first login or account settings update, including scanning a QR code and entering a verification code. SEE THIS DOC FOR USERS: [MFA Set Up Guidance](#)
3. The system shall prevent issuers from accessing their account until MFA is configured, but allow fans to proceed without MFA if they decline.
4. After successful primary login (social login for issuers, social, email/password, or passkey for fans), the system shall prompt users with MFA enabled to enter a time-based one-time password (TOTP) from their authenticator app.
5. The system shall provide users with 10-12 single-use backup codes during MFA setup, displayed on-screen and downloadable as a PDF, for recovery if the authenticator app is unavailable.
6. The system shall instruct users to store backup codes securely in a password manager or offline (e.g., printed in a safe place), not on their phone.
7. The system shall store backup codes encrypted in the user's account, accessible after primary login authentication.

8. The system shall allow users to use a backup code for a single login session if the authenticator app is unavailable, marking the code as used and invalidating it for future use.
9. The system shall permit users to generate a new set of backup codes if fewer than 3 remain, invalidating the old set.
10. If MFA verification fails, the system shall display an error (e.g., "Invalid MFA code. Please try again or use a backup code.") and allow up to 3 attempts before a 10-minute account lockout.
11. After a 10-minute lockout, if another 3 MFA attempts fail, the system shall enforce a 30-minute lockout.
12. After 9 total failed MFA attempts (3 sets of 3), the system shall require recovery via backup code or support intervention.
13. The system shall notify users via email after each lockout, warning of potential unauthorized access and providing recovery instructions (e.g., "Use a backup code or contact support").
14. The system shall offer an MFA recovery process for users who lose their authenticator app and backup codes, requiring a 30-minute email verification link sent to their registered email address.
15. The system shall require users to answer two security questions (e.g., "What was your first concert?" and "What's a nickname you had as a kid?") set during registration, after email verification, to complete MFA recovery.
16. The system shall ensure security questions are unique and obscure to reduce guessability, with answers stored hashed for security.
17. The system shall limit recovery attempts to 3 per day to prevent brute-force guessing of security questions.
18. If email verification or security questions fail, the system shall direct users to contact support for manual identity verification (e.g., confirming control of their registered social media account for issuers or email/social account for fans).
19. The system shall log all MFA setup, verification, recovery, and lockout events, including timestamps and IP addresses, to detect potential compromise attempts.
20. The system shall notify users via email of critical MFA events (e.g., setup completion, recovery attempts, lockouts).
21. The system shall provide in-app guidance (e.g., tooltips, video tutorial) for users setting up MFA, emphasizing its importance for account security.
22. The system shall ensure MFA prompts comply with WCAG 2.1 accessibility standards (e.g., screen reader support, keyboard navigation).
23. The system shall display an MFA encouragement prompt for fans who have not enabled MFA (e.g., "Enable MFA for added security!").

Physical Schema Design – Authentication, MFA & Recovery

DATA POINTS:

mfa_backup_code table

Field	Description	Type	Required	Notes
user_id	User identifier	UUID	Yes	
ID			Yes	Primary Key
mfa_backup_code	Hashed backup code	Text Typically 10~12	Yes	
status	Active/inactive (inactive = used or expired)	Enum	Yes	
mfa_backup_code_use_timestamp	Timestamp of when code is used	Timestamp	No	
mfa_backup_codes_issue_number	AUTO_INCREMENT for codes per user	Integer	Yes	
mfa_backup_code_sets_issued	AUTO_INCREMENT set of codes generated	Integer	Yes	
mfa_backup_code_generated_timestamp	New backup code generation timestamp	Timestamp	Yes	

DATA POINTS:

mfa_security_questions table

Field	Description	Type	Required	Notes
security_questions_set	Were security questions configured	Boolean	Yes	
security_questions_hashed	Stored hashed security Q&A	JSON	Yes	Hashed format
user_id		UUID		
created_at	When the questions are set	Timestamp		

DATA POINTS:

user_mfa table

Field	Description	Type	Required	Notes
user_id	User identifier	UUID	Yes	Foreign key to users table
issuer		Boolean	Yes	

mfa_enabled	Whether MFA is enabled	Boolean	Yes	Mandatory for issuers - mfa_setup_table
mfa_setup_completion_timestamp	When MFA was configured	Timestamp	Yes	mfa_setup_table
mfa_method	Authenticator method used	Enum	Yes	e.g., TOTP App (Google Authenticator) - mfa_setup_table
backup_codes_issued	Number of backup codes given	Integer	Yes	Transformation - mfa_backup_code table
backup_codes_remaining	Remaining unused backup codes	Integer	Yes	Transformation - mfa_backup_code table
backup_codes_last_generated	Timestamp when codes were last generated	Timestamp	Yes	Transformation - mfa_backup_code table
backup_codes_regenerated	Times new sets of backup codes were generated	Integer	Yes	Transformation - mfa_backup_code table
mfa_last_used_method	Last used method (app or backup code)	Enum	No	TOTP, Backup Code
mfa_id_last_used	mfa_id foreign key	UUID	Yes	FK - connects to mfa_log_table
mfa_failed_verification_attempts	Consecutive failed MFA attempts	Integer	Yes	Transformation - mfa_log table
mfa_lockout_count	Total lockouts triggered by failed MFA	Integer	Yes	Transformation - mfa_log table
mfa_lockout_active	Whether the account is currently locked	Boolean	Yes	Transformation - mfa_log table
mfa_lockout_expiry	When current lockout ends	Timestamp	No	Transformation - mfa_log table
mfa_last_ip	IP address of last MFA attempt	IP	Yes	Transformation - mfa_log table
mfa_last_device	Device/browser info of last MFA attempt	Text	No	Transformation - mfa_log table
mfa_recovery_initiated	Was MFA recovery triggered	Boolean	No	Transformation - mfa_log table
recovery_email_sent	Recovery email sent timestamp	Timestamp	No	Transformation - mfa_log table
security_question_attempts_today	Number of Q&A recovery attempts today	Integer	Yes	Max 3/day - Transformation from mfa_log table
mfa_manual_verification_required	Flag to escalate to support for verification	Boolean	No	

mfa_event_log	Logged history of MFA events	JSON	Yes	Timestamped events for audit
support_contacted	User contacted support	Boolean	No	For recovery or MFA issues

DATA POINTS:

mfa_log table

Field	Description	Type	Required	Notes
user_id		UUID	Yes	
user_type	Issuer/fan	Enum	Yes	
ID		UUID	Yes	PK
mfa_event	Description of event (TOTP, failure/success, etc.)	Text/JSON	Yes	
mfa_device	Device used for MFA	Text	Yes	
mfa_method	Which method was used		No	
login_session_id	Foreign key to login session ID		Yes	FK to login_session

login_session table

Field	Description	Type	Required	Notes
ID	session_id	UUID	Yes	Primary key
login_device	device_id, os, etc	Text	Yes	
login_user_agent	Device/browser of session	Text	Yes	
login_attempt_timestamp	Timestamp of login attempt	Timestamp	Yes	
login_success	Result of login attempt	Boolean	Yes	True if login succeeded
login_result_timestamp	Timestamp of login attempt result	Timestamp	Yes	
mfa_id	FK	UUID	No	Foreign key to mfa_log table
mfa_result	MFA verification status	Enum	No	Success, Failed, Timeout - from mfa_log table

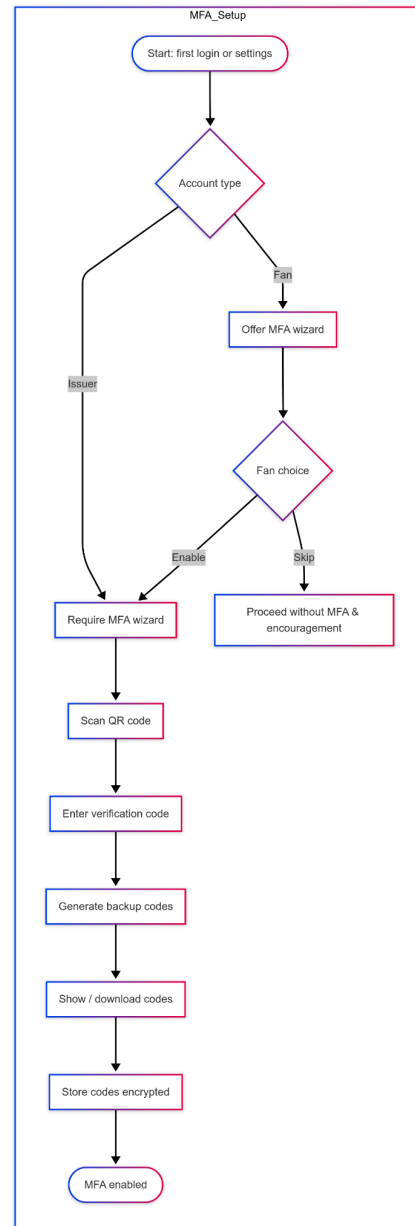
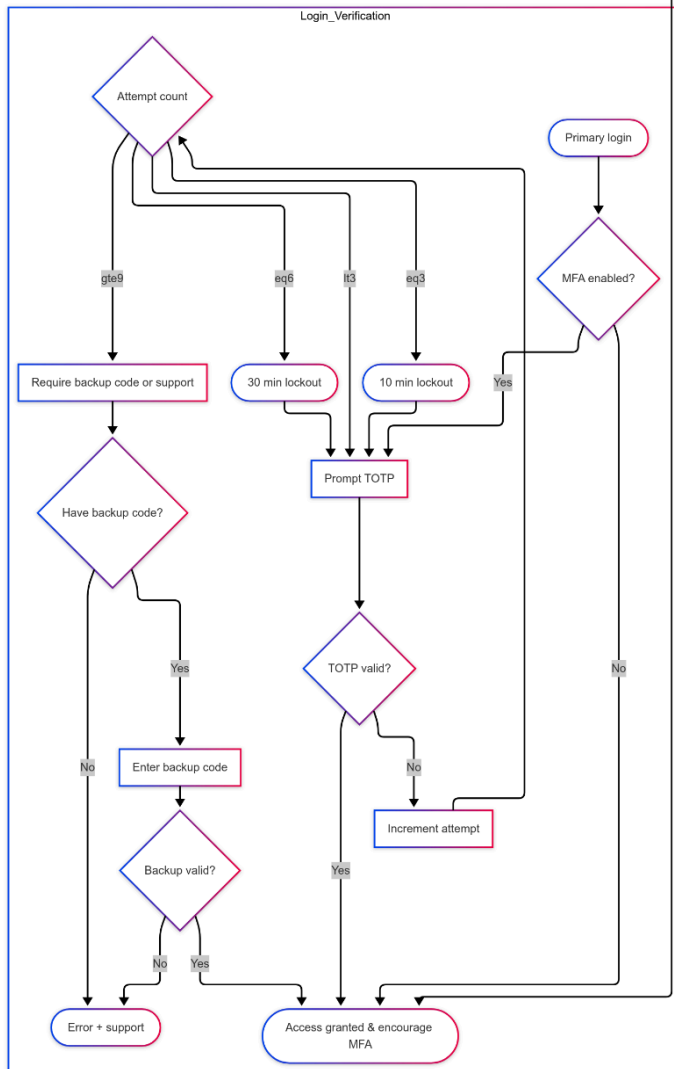
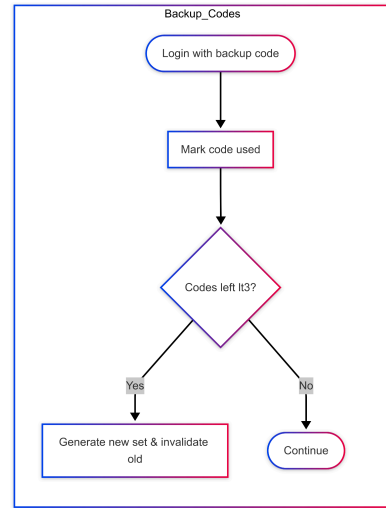
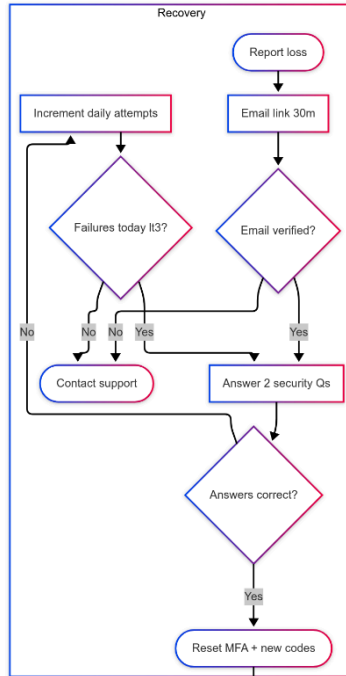
mfa_method_used	MFA method used	Enum	Yes if MFA	Authenticator App or Backup Code - from login_session
backup_code_used	Whether backup code was used	Boolean	No	True if used for login
login_location_detected	City/Region of login attempt	Text	No	GeoIP lookup
login_anomaly_detected	Anomaly flag (device/location)	Boolean	No	From login_anomaly table to be created

DATA POINTS:

user_login table

Field	Description	Type	Required	Notes
username	Unique ATHL username	Text	Yes	Used to identify user type and route login logic
user_id	user ID	UUID	Yes	
login_session_id	Unique login ID	UUID	Yes	
login_method	Login method used (email, social, passkey)	Enum	Yes	Fans: email/social/passkey; Issuers: social only
login_platform	Platform used for social login	Enum	Conditional	e.g., Twitter, Instagram, Google
login_user_agent	Device/browser of session	Text	Yes	from login_session
login_success	Successful, Failed, Lockout	Boolean	Yes	True if login succeeded
login_failure_reason	Reason for failed login	Text	No	Captured on failure (e.g., invalid password)
mfa_enabled	Is MFA enabled for this user	Boolean	Yes	Mandatory for issuers - from user_mfa
mfa_method_used	MFA method used	Enum	Yes if MFA	Authenticator App or Backup Code - from login_session
mfa_status	MFA verification status	Enum	Yes	Success, Failed, Timeout - from login_session
backup_code_used	Whether backup code was used	Boolean	No	True if used for login

session_token	Active session token	Text	Yes	Generated on successful login - from session_table to be created
session_expiry	Session expiration timestamp	Timestamp	Yes	



Session Management and Security

1. Upon successful login (including MFA for issuers or enabled fans), the system shall create a secure session with a session token, expiring after 30 minutes of inactivity or 24 hours of activity.
2. The system shall use HTTPS for all login-related communications to ensure data encryption.
3. The system shall implement rate-limiting, locking accounts after 5 failed login or MFA attempts for 10 minutes, with escalation to 30 minutes after another 5 failures.
4. The system shall log all login attempts (successful and failed), including timestamp, IP address, and user agent, for audit purposes.
5. The system shall monitor for social login anomalies (e.g., logins from new devices or locations) and trigger additional verification (e.g., email confirmation) if detected.
6. The system shall provide issuers with a dashboard showing recent login activity (e.g., timestamps, IP addresses, devices) to detect suspicious behavior.
7. The system shall encourage issuers to enable MFA on their social media accounts (e.g., Twitter/X, Instagram) via in-app prompts (e.g., “For added security, enable 2FA on your [platform] account”).

Password Recovery and Account Lockout

1. For email/password-based fan logins, the system shall provide a “Forgot Password” option, sending a secure reset link valid for 30 minutes to the registered email address.
2. The system shall offer a “Forgot Username” option, recovering the username via the registered email address.
3. For accounts locked due to failed login, the system shall notify the user via email with unlock instructions (e.g., secure link or support contact).

Accessibility and Usability

1. The login interface, including MFA prompts, shall comply with WCAG 2.1 accessibility standards (e.g., screen reader support, keyboard navigation).
2. The system shall offer 24/7 customer support (e.g., live chat, email) for MFA setup or recovery issues, prioritizing high-profile issuers.

Referral Engine (*This section Will be edited after Tom & Paul finalize new referral program)

- **FR-1:** Upon successful user sign-up, the system shall automatically generate a unique referral link for each user. (**EXPLORE NOT SPAMMY VERSIONS**)

- **Format:** `https://athl.live/refer?code=<unique_code>`.
- **<unique_code>:** Alphanumeric, 8-12 characters, globally unique.
 - **PAUL:** Should the unique code could be the user's ATHL handle?
- **FR-2:** The referral link shall be accessible to the user via:
 - Email confirmation post-sign-up.
 - Pop up window post sign-up
 - User Dashboard (Referral Dashboard section).
- **FR-3:** The referral link shall remain active indefinitely unless the user account is deactivated or until we end the referral program.
- **FR-4:** When a new user signs up via a referral link, the system shall:
 - Record the referral in the database, linking the new user to the referring user.
 - Validate the referral code to ensure it corresponds to an active user.
- **FR-5:** The system shall track the following for each referral:
 - Referring user ID.
 - Referred user ID.
 - Sign-up timestamp.
 - Location of referral link click (e.g. IG, X, YouTube) if possible.
 - IP Address
 - Token purchase(as a fan and issuer) and sales(as an issuer) of the referred user (for dynamic rewards) so far. (updated with activity)
- **FR-6:** The system shall prevent self-referrals (e.g., email, phone, or device).
- **FR-7:** The system shall award rewards as follows:
 - **Referring Fan:** 0.5% of the referred person's initial token purchases up to a total of \$20.
 - **Issuer:** 2% of referred issuer revenue from initial token sales up to a max of \$1,000 per issuer.
 - For an issuer that also purchases many tokens like a fan, the max referral payout for referring the issuer is still \$1,000.
- **FR-8:** Rewards shall be paid out in USDC to the user's Privy wallet
- **FR-9:** Rewards shall be dynamically updated based on the referred user's token purchase activity:
 - Trigger: Purchase event from the token purchase system.
 - Update frequency: Every hour if possible, or within 24 hours.

Referral Dashboard

- **FR-11:** The User Dashboard shall include a **Referral Dashboard** section displaying:
 - User's unique referral link (click-to-copy functionality).
 - Total number of referrals.
 - List of referred users with:
 - UserID
 - Sign-up date.
 - Classification: (Athlete, Creator, Fan)
 - Total Initial tokens purchases in USD (Fan reward).
 - Total Follower Token Sales in USD (Issuer reward).
 - Reward earned from each referral so far.
 - Total rewards earned (sum of fan and issuer rewards).
- **FR-12:** The Referral Dashboard shall support pagination for users with >20 referrals.
- **FR-13:** The Referral Dashboard shall be responsive, supporting desktop and mobile views.
- **Future Considerations - Referral system**
 - Fraud detection (e.g., detecting suspicious referral patterns).
 - Advanced analytics (e.g., referral funnel metrics).

DATA POINTS:

User_referral table

Field	Description	Type	Required	Notes
user_id	Unique user identifier	UUID	Yes	Used across all modules
ID	Primary Key	UUID	Yes	
referral_code	Globally unique referral code	Text	Yes	8-12 alphanumeric chars or ATHL handle
referral_link	Generated referral URL	Text	Yes	e.g., https://athl.live/refer?code=XXXX
referrer_user_id	User who made the referral	UUID	Yes if referred	
referred_user_id	User who was referred	UUID	Yes if referred	
referral_click_source	Click origin of referral	Text	No	e.g., IG, X, YouTube
referral_signup_timestamp	When referral user signed up	Timestamp	Yes	

referral_ip	IP address from referral link click	IP	Yes	
referral_reward_usdc	Reward paid to referrer (in USDC)	Float	Yes	Dynamic update based on issuer/fan, aggregated from backend table
referral_fan_purchases_usd	Token purchases by referred fan	Float	Yes	Aggregated from backend table
referral_issuer_sales_usd	Token sales by referred issuer	Float	Yes	Aggregated from backend table

Waiting List Creation and Management

- **FR-1:** The system shall allow users signing up as **issuers** to be automatically added to a waiting list for launching their own tokens, while granting them immediate access to the platform as **fans** to purchase tokens and refer others.
- **FR-3:** The system shall notify issuers via email upon being added to the waiting list for token launching, including:
 - Confirmation of their waiting list status for launching tokens.
 - Confirmation of immediate access to purchase tokens as a fan.
 - Reminding them to refer others to the platform along with their unique referral codes.
- **FR-5:** The system shall support phased onboarding of issuers from the waiting list for launching tokens:
 - **Phase 1:** Approve and onboard the first 100 issuers manually
 - **Phase 2:** 1–2 weeks later, approve and onboard 100–500 issuers.
- **FR-6:** The system shall allow administrators to manually approve issuers for token launching.
- **FR-7:** Upon approval for token launching, the system shall:
 - Email the issuer 2-3 days before launch with instructions to access token launching features.
 - Remove the issuer from the waiting list and update their account status to “Active Issuer” (while retaining their fan role for token purchasing).
- **FR-8:** The system shall maintain a record of each issuer’s waiting list status for token launching (e.g., Pending, Approved, Denied) and approval timestamp.

DATA POINTS:

waitlist_table table

Field	Description	Type	Required	Notes
-------	-------------	------	----------	-------

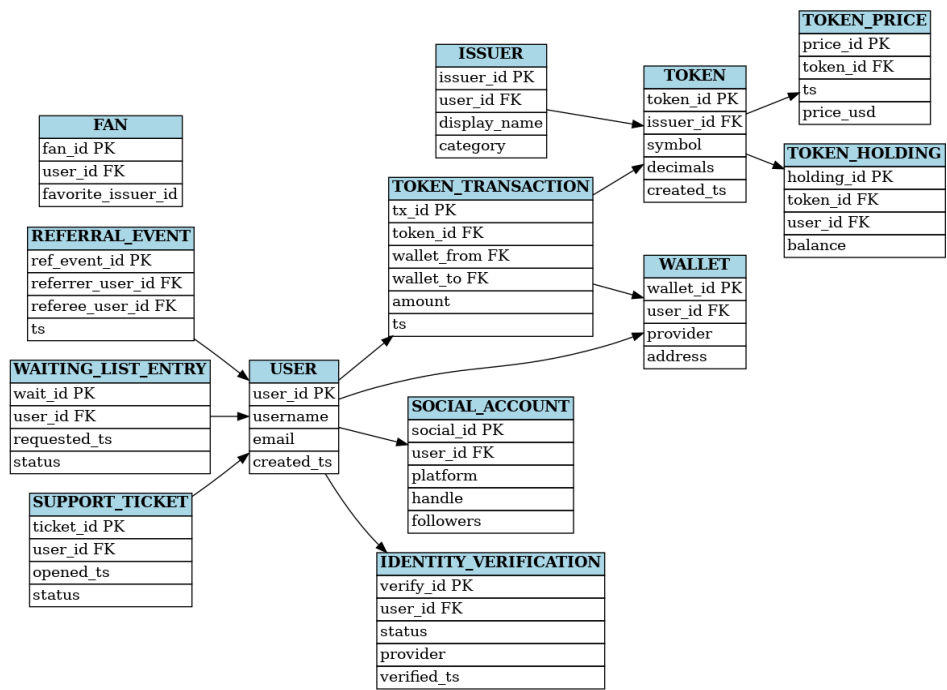
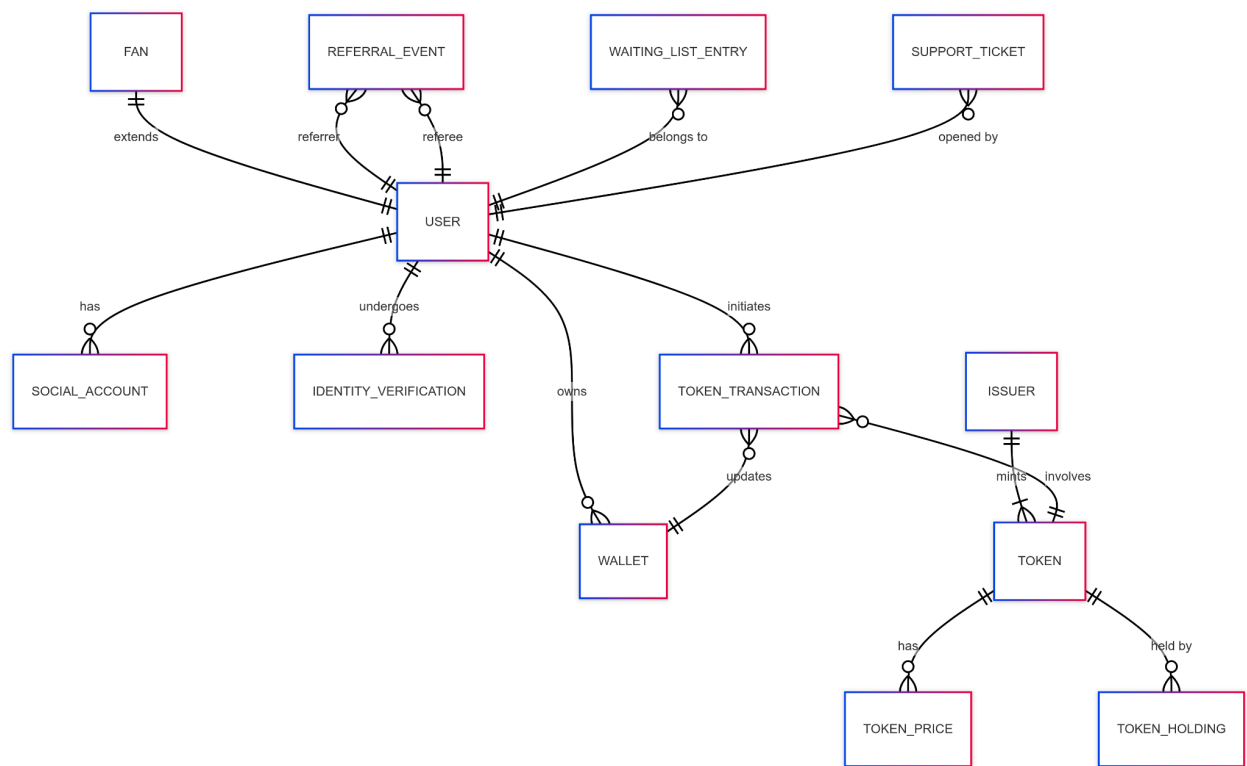
ID	Primary key	UUID	Yes	
waitlist_status	Is the user now a token issuer?	Boolean	Yes	Pending, Approved, Denied
created_at	Timestamp of user joining the waitlist	Timestamp	Yes	
updated_at	Timestamp of waitlist decision (Approved, Denied)	Timestamp	Yes	Same as created_at if pending
waiting_list_status	Token launch status	Enum	Yes	Pending, Approved, Denied
waiting_list_joined	Date user joined the waiting list	Timestamp	Yes	
waiting_list_approved_time	Approval timestamp	Timestamp	No	
approval_phase	Phase of approval (1 or 2)	Enum	No	
admin_notes	Manual comment on user	Text	No	
admin_action_log	Audit log of admin activity	JSON	Yes	Timestamps + admin IDs

Internal Admin Page

- **FR-9:** The system shall provide an **internal admin page** accessible only to authorized team members to manage and review the issuer waiting list for token launching.
- **FR-10:** The admin page shall display the following information for each issuer on the waiting list:
 - User ID.
 - Highest social account, follower #s, and handle
 - Sign-up timestamp.
 - Demographic data (e.g., name, email, age, location).
 - Issuer type(athlete-creator)
 - athlete sport or creator type.
 - Number of referrals generated and accrued rewards.
 - Total token purchase amount
 - Waiting list status for token launching (Pending, Approved, Denied).
- **FR-11:** The admin page shall support the following functionalities:
 - **Filtering:** By sign-up date, demographic data, referral activity, token purchase activity, or waiting list status.
 - **Sorting:** By sign-up timestamp, name, highest follower count, referral count, token purchase amount, Issuer type(Athlete/creator), sport or creator type
 - **Search:** By email, name, user ID, sport or creator type

- **Pagination:** For lists exceeding 50 issuers per page.
 - **Bulk Actions:** Approve or deny multiple issuers for token launching at once (e.g., select 100 issuers for Phase 1 onboarding).
- **FR-12:** The admin page shall include summary metrics to evaluate waiting list performance, including:
 - Total issuers on the waiting list for token launching.
 - Number of issuers approved vs. pending for token launching.
 - Average wait time (from sign-up to approval for token launching).
- **FR-13:** The admin page shall allow administrators to export waiting list data as a CSV file, including all displayed fields
- **FR-16:** The admin page shall display referral performance for each issuer (e.g., number of referrals, total rewards earned) to support evaluation and prioritization for token launching approval.
- **FR-17:** The admin page shall include a notes or comments field for each issuer, allowing administrators to document observations or reasons for approval/denial for token launching (e.g., “High token purchases as fan, prioritize”).
- **FR-18:** The system shall log all admin actions (e.g., approvals, denials, notes) with timestamps and the acting administrator’s ID for audit purposes.

Entity Relationship Diagram



Fan Dashboard

- **FR-1:** The Fan Dashboard shall be the first page displayed to authenticated users (fans and issuers acting as fans) upon logging into the ATHL platform.
- **FR-3:** The dashboard shall display the following sections in order:
 - Wallet balances (Privy and ATHL wallets).
 - Line chart for total balance growth.
 - ATHL wallet details for follower token holdings.
 - Leaderboards for issuers.
 - Suggested issuer tokens.
 - Search and filter tool.
 - Watch list.
- **FR-4:** The dashboard shall display the user's wallet balances:
 - **Privy Wallet:** Show the balance of non-ATHL assets (e.g., fiat or other cryptocurrencies) in USD equivalent.
 - **ATHL Wallet:** Show the total value of off-chain follower token holdings in USD equivalent.
- **FR-5:** The dashboard shall display the combined total balance of all holdings (Privy + ATHL wallets) in USD equivalent.

Total Balance Line Chart

- **FR-6:** The dashboard shall include a line chart showing the growth of the user's total balance (Privy + ATHL wallets) over time, styled similarly to Robinhood's asset change chart.
- **FR-7:** The line chart shall support user-selectable time frames:
 - 24 hours.
 - 1 week.
 - 1 month.
 - All time (since account creation).
- **FR-8:** The line chart shall update dynamically when the user changes the time frame, fetching historical balance data for the selected period.
- **FR-9:** The line chart shall display positive growth in green and negative growth in red, with percentage change shown for the selected time frame.
- **FR-10: ATHL WALLET HOLDINGS** The dashboard shall display a list of the user's follower token holdings in the ATHL wallet, with the following details for each token:
 - **Issuer Name:** User ID, first name, and last name of the issuer.

- **Current Price:** Token price in USD equivalent.
- **Daily % Change:** Percentage change in token price over the last 24 hours, displayed in green (up) or red (down).
- **Quantity Held:** Number of tokens owned by the user.
- **Total Value:** Value of the user's holding for that token (quantity × current price).
- **FR-11:** The list shall support pagination or scrolling for users holding more than 10 tokens.
- **FR-12:** Each token entry shall include a clickable issuer name linking to the issuer's specific page

Leaderboards

- **FR-13:** The dashboard shall display **LEADERBOARDS** showcasing top-performing issuers in the following categories:
 - **Highest Price:** Issuers with the highest current token price.
 - **Most Sales:** Issuers with the highest number of token sales in the last 24 hours or 1 week (user-selectable).
 - **Most Money Raised:** Issuers who have raised the most total funds through token sales.
- **FR-14:** Each leaderboard shall display the top 10 issuers by default, with an option to view up to 20.
- **FR-15:** Each leaderboard entry shall include:
 - Issuer name (User ID, first name, last name).
 - Relevant metric (e.g., token price, number of sales, total funds raised).
 - A clickable link to the issuer's specific page (see FR-24).
 - An "Add to Watch List" button (see FR-22).
- **FR-16:** The leaderboards shall include a filter to display issuers by:
 - **Creator Type:** E.g., Podcaster, Musician, artist.
 - **Sport:** E.g., basketball, soccer, tennis (for athlete creators only).
- **FR-17:** The leaderboards shall update dynamically when filters or time frames (for sales leaderboard) are applied.

Suggested Issuer Tokens

- **FR-18:** The dashboard shall display a section of **SUGGESTED ISSUER TOKENS** to buy, showing up to 5 issuers by default.

- **FR-19:** Suggested issuers shall be selected based on criteria such as recent sales volume, price growth, or platform popularity (algorithm TBD).
- **FR-20:** Each suggested issuer entry shall include:
 - Issuer name (User ID, first name, last name).
 - Current token price.
 - Daily % change in price (green for up, red for down).
 - A clickable link to the issuer's specific page (see FR-24).
 - An "Add to Watch List" button (see FR-22).

Search and Filter Tool

- **FR-21:** The dashboard shall provide a **SEARCH AND FILTER TOOL** allowing users to find issuers based on the following criteria:
 - **Creator Type:** E.g., athlete, influencer, artist.
 - **Sport:** E.g., basketball, soccer, tennis (for athletes).
 - **Position:** E.g., quarterback, point guard (for athletes in specific sports).
 - **School:** E.g., college or university name (for athletes).
 - **Team Name:** E.g., professional or amateur team (for athletes).
 - **Minimum Follower Count:** From the issuer's primary social media platform.
 - **Maximum Follower Count:** From the issuer's primary social media platform.
- **FR-22:** The search tool shall include a text input for partial matches on issuer names (User ID, first name, last name).
- **FR-23:** The search results shall display a list of matching issuers, each including:
 - Issuer name (User ID, first name, last name).
 - Current token price.
 - Daily % change in price (green for up, red for down).
 - A clickable link to the issuer's specific page (see FR-24).
 - An "Add to Watch List" button (see FR-27).
- **FR-24:** The search results shall support pagination for lists exceeding 10 issuers.
- **FR-25:** Clicking an issuer's name in the leaderboards, suggested tokens, or search results shall navigate to an **ISSUER-SPECIFIC PAGE**
- **FR-26:** The issuer-specific page shall allow users to return to the Fan Dashboard via a back button or navigation link.

Watch List

- **FR-27:** The dashboard shall allow users to create and manage a **“WATCH LIST”** of issuers they want to track.
- **FR-28:** The watch list section shall display a list of issuers added by the user, including:
 - Issuer name (User ID, first name, last name).
 - Current token price.
 - Daily % change in price (green for up, red for down).
 - A clickable link to the issuer’s specific page (see FR-24).
 - A “Remove from Watch List” button.
- **FR-29:** Users shall be able to add issuers to the watch list via:
 - An “Add to Watch List” button on leaderboards, suggested tokens, or search results (without visiting the issuer-specific page).
 - An “Add to Watch List” button on the issuer-specific page.
- **FR-30:** The watch list shall support up to 50 issuers per user and display pagination or scrolling for lists exceeding 10 issuers.
- **FR-31:** The watch list shall persist across user sessions, retaining the user’s selections until manually removed.
- **FR-32:** All prices and values displayed on the dashboard shall be in USD equivalent, updated in real-time or at least every 1 minute.

Referral Dashboard

- **FR-33:** We shall give the user full referral progress information as discussed in the **Referral Engine section: FR-11**

Issuer Specific Purchase Page

Page Access and Layout

- **FR-1:** The Issuer Specific Purchase Page shall be accessible by clicking an issuer’s name from the Fan Purchase Dashboard (e.g., leaderboards, search results, watch list) or via a direct URL (e.g., https://athlplatform.com/issuer/<issuer_id>).
- **FR-2:** The page shall be responsive, supporting desktop and mobile views, with a consistent user experience across devices.
- **FR-3:** The page shall display the following sections in the specified order, with layout (e.g., left, right, middle) to be determined by the designer:
 - Token price line chart.
 - Current state of token sales.

- Issuer profile information.
- Privy wallet balance and payment options.
- Token purchase window.
- Purchase history.

Token Price Line Chart

- **FR-4:** The page shall include a **line chart** showing the issuer's token price changes over time, styled similarly to Robinhood's price charts.
- **FR-5:** The line chart shall support user-selectable time frames:
 - 1 day.
 - 1 week.
 - 1 month.
 - All time (since the token's launch).
- **FR-6:** The line chart shall:
 - Display price in USD equivalent on the y-axis and time on the x-axis.
 - Show percentage change for the selected time frame, in green (up) or red (down).
 - Update dynamically when the user changes the time frame, fetching historical price data.
- **FR-7:** The line chart shall include tooltips showing the price and timestamp when hovering over data points.

Current State of Token Sales

- **FR-8:** The page shall display the current state of the issuer's token sales, including:
 - **Total Amount Raised:** Total funds raised in USD equivalent.
 - **Number of Tokens Sold:** Total tokens purchased by users.
 - **Current Price:** Token price in USD equivalent, updated in real-time or every 5 minutes.
- **FR-9:** The sales data shall be clearly visible to ensure users can quickly assess the token's performance.

Issuer Profile Information

- **FR-10:** The page shall display the issuer's profile information, including:
 - Full name (first and last name).
 - ATHL username.

- Creator type (e.g., athlete, influencer, artist) and subtype (e.g., basketball player, musician).
- Verified social media accounts (e.g., X, Instagram) with follower counts for each platform. Clickable
- ID Verification status: A checkmark icon indicating if the issuer's ID is verified.
- Bio statement (text box).
- Perks offered to token holders (text box).
- Links to personal videos or recent social media post clips (clickable, opening in a new tab or embedded player).
- **FR-11:** Social media links shall be clickable, directing to the issuer's verified profiles, with follower counts updated at least daily.

Privy Wallet Balance and Payment Options

- **FR-12:** The page shall display the user's Privy wallet balance in USD equivalent, showing available funds in USDC and SOL for token purchases.
- **FR-13:** The Privy wallet balance shall be clearly visible to help users assess available funds.
- **FR-14:** The page shall indicate that users can purchase tokens using:
 - Privy wallet (USDC or SOL).
 - Stripe (fiat payment via credit card or other Stripe-supported methods).
- **FR-15:** The payment options shall be selectable via buttons or a dropdown in the token purchase window (see FR-18).

Token Purchase Window

- **FR-16:** The page shall include a token purchase window styled similarly to Robinhood's trading interface, allowing users to purchase tokens by:
 - **Number of Tokens:** Input field for the desired quantity of tokens.
 - **USD Amount:** Input field for the desired purchase amount in USD, automatically calculating the equivalent number of tokens based on the current price.
- **FR-17:** The purchase window shall display the estimated total cost (in USD) and number of tokens based on user input, updating dynamically as the user adjusts the quantity or amount.
- **FR-18:** The purchase window shall include a verification step after the user submits their order, requiring:
 - Confirmation of the order details (tokens, USD amount, payment method).

- Acceptance of any necessary legal agreements (e.g., terms of service, risk acknowledgment) via checkbox or button.
- **FR-19:** The purchase window shall support two payment methods, selectable via buttons or dropdown:
 - **Privy Wallet:** Using available crypto funds (USDC or SOL).
 - **Stripe:** Using fiat payment (e.g., credit card, processed via Stripe).
- **FR-20:** The purchase window shall validate the user's Privy wallet balance (see FR-12) before allowing a Privy wallet purchase, displaying an error if funds are insufficient.
- **FR-21:** Upon successful purchase, the page shall:
 - Display a green checkmark or success indicator (e.g., "Purchase Successful") for 5 seconds.
 - Update the user's ATHL wallet follower token balance in real-time.
 - Automatically refresh the purchase history section

Purchase History

- **FR-22:** The page shall display a **purchase history** section listing the user's previous transactions for the issuer's tokens, including:
 - **Date:** Timestamp of the purchase.
 - **Price:** Token price at the time of purchase (in USD equivalent).
 - **Purchase Amount:** Number of tokens purchased and total USD cost.
- **FR-23:** The purchase history shall support pagination or scrolling for users with more than 10 transactions.
- **FR-24:** The purchase history shall update automatically after a successful purchase, reflecting the new transaction in real-time.

General Requirements

- **FR-25:** All prices and values displayed on the page shall be in USD equivalent, updated in real-time or at least every 1 minute.
- **FR-26:** The page shall include a button to add the issuer to the user's watch list, linking to the watch list functionality on the Fan Purchase Dashboard
- **FR-27:** The page shall allow users to return to the Fan Purchase Dashboard via a back button or navigation link.

Issuer Admin Pages

1. Issuer Profile Header

- Display the issuer's name and issuer classification (e.g., athlete, creator) at the top of the page.
- Ensure the issuer classification information (e.g. Athlete, Quarterback, Football, UCLA) is pulled from the issuer's profile data and is non-editable in this view.

2. Total Token Sales Revenue Line Chart

- Display a line chart below the profile header showing total token sales revenue (sum of initial token sales and trading fees).
- X-axis: Time (default: daily data points for the past 30 days, other options are 1 day, 1 week, 6 months, 1 year and all time).
- Y-axis: Revenue in USD.
- Include a toggle to switch between:
 - **Daily Totals:** Revenue per day.
 - **Aggregate Totals:** Cumulative revenue over time.
- Update table data in real-time

3. Token Metrics Table

- To the right of the total token sales revenue chart, display a small table with the following columns:
 - **Current Price:** Current trading price of the issuer's tokens (in USD).
 - **Coins Sold:** Total number of tokens sold (initial sales - this is the amount of tokens in float).
 - **Coins Available:** Number of tokens still available for sale.
 - **24-Hour Turnover Percentage:** amount of token transactions / price of token * number of tokens in float (initial sales).
- Update table data in real-time

4. Revenue Breakdown Line Charts

- Below the total revenue chart, display two line charts:
 - **Revenue from Initial Token Sales:**
 - X-axis: Time (default: daily data points for the past 30 days, other options are 1 day, 1 week, 6 months, 1 year and all time).
 - Y-axis: Revenue from initial token sales in ATHL tokens.
 - **Revenue from Secondary Sales Trading Fees:**
 - X-axis: Time (default: daily data points for the past 30 days, other options are 1 day, 1 week, 6 months, 1 year and all time).
 - Y-axis: Revenue from trading fees in ATHL tokens.
- Include a toggle for each chart to switch between:
 - **Daily Totals:** Revenue per day.

- **Aggregate Totals:** Cumulative revenue over time.
- Provide zoom and pan functionality for both charts.

5. Expected Proceed Payments Table

- Below the revenue breakdown charts, display a table showing expected distribution payments over the next 12 months.
- **Table Structure:**
 - **Columns:** Year (current year), followed by each month (e.g., Jan, Feb, ..., Dec).
 - **Rows:** One row per year, starting with the current year.
 - **Cells:** Expected distribution payment amount (in USD) for each month.
- Data should reflect projections based on token sales and platform rules.
- Highlight the current month's cell for clarity.

6. Editable Issuer Profile Details

- Below the payments table, display a form containing all issuer profile details that the issuer filled out during account creation, this should include:
 - Athlete or creator classification (e.g., athlete, musician).
 - Sports (if applicable, e.g., track, basketball).
 - Schools (e.g., high school, college).
 - Current team (if athlete)
 - Mission (text description of the issuer's goals).
 - Benefits to buyers (text description of token holder perks).
 - Links (e.g., website, portfolio).
- All fields are editable except for social handles (e.g., X handle, Instagram), which are displayed as read-only.
- Include an "Edit" button to create and submit changes and a "Cancel" button to discard edits.
- Validate input fields (e.g., valid URLs for links, non-empty mission text) before saving.
- Display a success message on save or an error message if validation fails.

7. Follower Token Holder Table

- Below the profile details form, display a table with information about the issuer's follower token holders.
- **Table Columns:**
 - **User ID:** Token holder's platform User ID.
 - **Email:** Token holder's email address.
 - **Quantity Held:** Number of tokens currently held.
 - **Date Purchased:** Date of the most recent token purchase.
 - **Benefits Received:** List of benefits provided to the token holder (e.g., exclusive content, event access).

- Include sorting options for each column (e.g., sort by quantity held, date purchased).
- Provide a filter to search by User ID or email.
- Paginate the table with 25 entries per page and “Next”/“Previous” buttons.

8. Fantasy Teams Tables

- Below the token holder table, display a table for each Fantasy Team the issuer is part of (as organizer, leader, or member).
- **Table Header:** Display the team name as the heading for each table.
- **Table Columns** (per team, as defined in Fantasy Teams requirements):
 - **User ID:** Team member’s User ID.
 - **Email Address:** Team member’s email.
 - **Country:** Team member’s country (from profile).
 - **Time Zone:** Team member’s time zone (from profile).
 - **Athletic Sport/Creator Type:** Team member’s primary sport or creator type (from profile).
 - **Confirmation Status:** Status of invitation (Pending, Yes, No, No Response).
 - **Remove Button:** A “Remove” button or minus sign to remove a team member (visible only to organizers).
- **Edit Permissions:**
 - **If the issuer is the team organizer:** The table is fully editable, including:
 - Add new members via a “+” button in the upper-right corner, opening the team creation form (as per Fantasy Teams requirements).
 - Remove members via the “Remove” button, with a confirmation dialog (“Are you sure you want to remove [User ID]?”).
 - Edit existing team details (e.g., purpose, coaching frequency) via an “Edit Team” button.
 - **If the issuer is a team leader but not the organizer:** The table is read-only, with no add/remove/edit options.
 - **If the issuer is a team member:** The table is read-only, with no add/remove/edit options.
- Update tables in real-time when team data changes (e.g., new member added, status updated).
- Include sorting options for columns (e.g., sort by status, country) and pagination (25 entries per page).

Developing Fantasy Teams

PM Notes:

- Let’s start out this concept by focusing on track/running, which is easy to get player data around, and accessible globally
- Sports icons (we can term them “Captains” or “team owners”) such as Adrian Peterson have the ability to create their team (i.e. ATHL’s take on fantasy)

- Team is composed of players in various events such as 100m, 200m, 400m, 1000m, 1500m, 4x100 relay, etc.
- Players can be in any age brackets for different events, but we want to start with a focus on under 14-year olds (community)
- ATHL keeps control (we need a toggle in admin to enable) of who can be made a Captain (maybe we have some criteria where issuers can qualify to be a Captain).
- The Team issues its fan token under team name (proceeds used for things like travel, shirts, coaching, gym, and equipment)
- Each player on the team must have their own fan token in order to be recruited to a team
- Teams can be comprised of remote athletes or locally, and are coached by Captains over Zoom
- Adrian can challenge another Captain such as Ray Lewis to any event or series of events (either one-off, or over a period of time)
- That challenge would use an email notification, as well as a callout on X (so we get the public excitement/involvement)
- All challenges appear on the ATHL home page for fans to see, so they can buy the fan tokens of players or the teams
- [Later we can allow fans to bet, and make other wagers, on the events using ATHL coin]
- The event outcomes (times for each player) are self-reported and based on a staking system where the team token or the captain's token stakes the validity of data-times
- Note (Sys Req): We want to push all messaging to X so that it's public content and fans can interact there
- We need to think about how to highlight this and bring in all the competitive elements, trash talking between Captains, increasing public/fan support for their favorite Captain by adding to trash talk, etc.

The Fantasy Teams of Issuers feature allows issuers to form teams led by team leaders (e.g., often by retired professional athletes, but can be any issuer and this can be for any types of creators as well) to compete in athletic or creative competitions. Once a team is formed, they will launch their own token, and both team leaders and members must stake tokens to participate. The feature includes team creation, invitation management, competition organization, and staking enforcement to ensure fair play.

1. Team Creation Form

- **Location:** Issuer page within the ATHL trading platform.
- **Form Fields:**
 - **Team Leader User IDs:** Input for 1–5 team leader User IDs (REQ, validated against existing issuer account to make sure they exist).
 - **Team Name:** (REQ, 1-35 Characters, alphanumeric, validated to not be the same name of another team)

- **Invited Issuer User IDs:** Input from 1 to 30 issuer User IDs to invite (REQ. validated against existing issuer accounts to make sure they exist).
- **Purpose, Goals, Objectives:** Text area for describing the team's purpose, goals, and objectives and why other issuers should join. (REQ, max 1000 characters, this can include links to videos or websites).
- **Coaching Meeting Frequency:** Numeric input for expected coaching or team meetings per week (min: 1, max: 14).
- **Competition/Challenge Frequency:** Numeric input for expected competitions per month (min: 1, max: 15).
- **Hourly Commitment:** Numeric input for expected weekly hourly commitment for issuer team members (min: 1, max: 50).
- **Response Deadline:** Date picker for when invitees must respond (must be at least 24 hours and max 10 days).
- **Contact Information:** Input for team leader's email and/or WhatsApp number for team member inquiries.
- **Submit Button:** Submits the form and triggers invitation notifications.
- **Behavior:**
 - On submission, save the team data in the backend and trigger individual invitation messages via email backend, to each invited issuer.

2. Invitation System

- **Notification Delivery:**
 - Send an email to each invited issuer including the additional team leaders, via ATHL's backend email service.
 - Email content includes all form data (team leader IDs, purpose, goals objectives, links, coaching frequency, organizers contact information, etc.) along with the question "Would you like to accept this invitation?" with two button links: **Yes** and **No**.
- **Response Handling:**
 - Clicking **Yes** or **No** opens a pop-up window confirming the response was received.
 - Record the response in the backend and update the team organizer's team status table.
 - Send a confirmation email to the team organizer with the invitee's response (User ID, response: Yes/No).
- **Timeout:**
 - If an invitee does not respond by the deadline, mark their status as "No Response" in the organizer's table and notify the organizer via email.

3. Teams Page for Issuers

- **Page Structure:**

- Dedicated “Teams” tab or section on the issuer’s dashboard.
- Displays a table for each team created by the organizer, with the team name as the table title.
- **Table Columns** (per team):
 - **User ID:** Issuer’s User IDs, starting with the organizer, then other team leaders, then invited issuers. These are all links that connect to the issuer’s purchase page
 - **Email Address:**
 - **Country:**
 - **Time Zone:** Invitee’s time zone (from profile).
 - **Age of issuers**
 - **Athletic Sport/Creator Type:** Invitee’s primary sport or creator type (from profile).
 - **Confirmation Status:** Status of invitation (Pending, Yes, No).
 - **Notes:** This is a place where the organizer can add notes about each issuer. There should be an edit button over this section to make ongoing edits.
 - **Remove Button:** A “Remove” button or minus sign to remove a team member. If a team member is removed, then that team member should receive an email letting them know they have been removed.
- **Table Actions:**
 - **Add Members:** A “+” button in the upper-right corner opens the team creation form with all fields editable. The “Invited Issuer User IDs” field is blank for adding new members.
 - **Remove Members:** Clicking “Remove” next to a team member prompts a confirmation dialog (“Are you sure you want to remove [User ID]?”). On confirmation, remove the member from the team and notify them via email.
 - **Notes:** This is a place where the organizer can add notes about each issuer. There should be an edit button over this section to make ongoing edits.
- **Table Updates:**
 - Real-time updates to the table when an invitee responds or a member is added/removed.
 - Sorting and filtering options for columns (e.g., sort by status, filter by country).
- **Team Table for all team issuers**
 - For each issuer, in their token issuer pages, add the team table for each team they are participating in. This table will not be editable. Edit privileges will only go to the team organizer. But we want each team member to have the complete team information.

4. Token Staking to validate Team Creation and ability to Compete

- **Staking Requirements:**
 - Team leaders must stake 5,000 of their own tokens per team.

- Team members must stake 100 tokens each upon accepting the invitation.(???)
- **Implementation:**
 - On team creation, the issuer will need to transfer 5,000 of their own tokens per leader and lock the tokens in an escrow system.
 - Store staking details in the backend (team ID, user ID, staked amount, timestamp).
- **Cheating Penalty:**
 - If a team is flagged for cheating (via a separate moderation process), forfeit all staked tokens to a designated platform wallet.
 - Notify all team members and leaders via email when tokens are forfeited.
 - The team can remain a team if caught cheating, but in their record, it will be recoded as a loss AND that they cheated.
- **Token Release:**
 - If a team breaks apart or a member is removed without cheating, release their staked tokens back to their wallet.

5 Team Token Creation

- **Token Generation:**
 - Once one or more team members accept the invitation to the team, add a “Mint Team Token” Below the table of invited team members. This would need to be at least one team member to accept, not just 1 or 2 team leaders.
 - Once clicked, a token launch form should appear including the following items
 - Team name (Autofilled from the table)
 - Token Launch Data(REQ, Data and Time)
 - Desired Amount to be raised (REQ: dropdown From 2M to 10M in increments of 1 Million)
 - The team leader who created the team is the only one who can launch the token. All team members will have access to the team page showing token sales information and expected proceed pay-outs.
 - The team leader who filled out the team creation form is the only one who has control of the team wallet.
 - In the last row of the team table, in one row, add a link to the team token purchase page.
- **Distribution:**
 - Initial token supply is minted to a team wallet controlled by the team leader.
 - Incoming Team proceeds will be managed by the Team Organizer leader.
- **Integration:**
 - Integrate with the platform’s token trading system to allow team tokens to be traded like other issuer tokens.

5. Team Token Purchase Page

- The team leaders and all members will be listed on this page and will have the same format as all other issuer pages.
- Next to each team member user ID, will be their verified social handles.
- It will include the the team's purpose, goals, and objectives.
- It will have a listing of all competition requests that the team leaders have made as well as competition invites from other teams.
- It will have a calendar listing of all upcoming competitions with other teams.
- It will have a history of previous competitions, all participating teams and team members and the results, in reverse chronological order.

Team Competitions

The Team Competitions section will be added to the existing Fan Dashboard, below the Recommended Issuers section. This section enables fans to view upcoming team competitions, including team details, competition dates, event types, and fantasy betting probabilities similar to Polymarket. Fans can place bets on competition outcomes via an order book, with probabilities dynamically updating based on supply and demand. Additionally, a leaderboard will display the most successful teams based on their win history.

Competition Organization

- **Challenge System:**
 - Team leaders can issue challenges to other teams via a "Challenge" button on the Teams token purchase page.
 - Challenge form includes:
 - Opposing team ID (REQ, dropdown of the teams the team leader is leading).
 - Competition type (REQ, Free form text , such as 100-meter sprint).
 - Proposed date and time (REQ, must be at least 72 hours in the future, include time zone).
 - Rules/parameters REX (text area, max 1,000 characters).
 - Submit button to send the challenge.
 - On submission, send an email to the opposing team's leader(s) with challenge details and Accept/Decline buttons.
 - Update both team's token purchase pages by adding this challenge to the challenge requests of the team that invited the challenge as well as adding to the challenge invites of the team the received the challenge invite.
- **Response Handling:**

- If the opposing team accepts, notify both teams and log the competition in the backend. Update the upcoming competitions table of both team's token purchase pages.
- If declined or no response within 72 hours, mark the challenge as "Declined" and notify the challenger. Remove the challenge from the requested and invited sections of each team's purchase pages.
- **Competition Tracking:**
 - Store competition details (teams, type, date, rules, outcome) in the backend.
 - Display upcoming and past competitions on the Teams page for each team.

2 Team Competitions Section Layout

- **Location:** Fan Dashboard, below the Recommended Issuers section.
- **UI Design** (inspired by Polymarket):
 - Display a list of upcoming competitions in a card or table format.
 - Each competition card/table row includes:
 - **Competing Teams:** Names / User IDs of the two teams (e.g., "PetersonSprintTeam vs. SmithRunners").
 - **Date of Competition:** Date and time of the event (e.g., "June 10, 2025, 3:00 PM UTC").
 - **Competition Type:** Type of event (e.g., "100-meter sprint").
 - **Fantasy Betting Probabilities:** Percentage likelihood of each team winning (e.g., Team A: 60%, Team B: 40%).
 - **Bet Button:** A "Place Bet" button to open the betting interface.
- **Sorting and Filtering:**
 - Allow sorting by date (ascending/descending) or competition type.
 - Provide a filter for competition types (e.g., show only 100-meter sprint events).
- **Pagination of competition sections:**
 - Display 6-10 competitions per section with "Next" and "Previous" buttons for navigation.

Fantasy Betting System

- **Betting Interface:**
 - Clicking "Place Bet" opens a modal or sidebar with:
 - Team selection dropdown (e.g., Team A or Team B).
 - Bet amount input (numeric, in ATHL tokens, min: 1 token).
 - Current probability display for the selected team.

- Order book summary (top 5 buy/sell orders for each team, showing price and quantity).
 - “Market Bet” button to bet at the current probability price.
 - “Limit Bet” button to place a custom order in the order book (specify price and quantity).
 - “Cancel” button to close the modal.
- **Order Book Mechanics** (Polymarket-style):
 - Maintain a real-time order book for each competition outcome (e.g., Team A wins, Team B wins).
 - Fans can place:
 - **Market Bets:** Instantly executed at the current probability-based price.
 - **Limit Bets:** Added to the order book at a specified price, executed when matched.
 - Probabilities update dynamically based on order book activity:
 - Probability is the current midpoint of the buy - sell spread.
 - Update probabilities after every transaction.
 - Store all bets in the backend with user ID, team, amount, price, and timestamp.
- **Bet Settlement:**
 - After a competition concludes, mark the winning team in the backend (via admin or oracle input).
 - Distribute payouts to fans who bet on the winning team based on their bet amount and final probability.
 - Forfeit bets on the losing team.
 - Notify users via email of their bet outcome (win/loss, payout amount).
 - There is a 0.5% fee taken on all winning payouts.
- **Token Integration:**
 - Bets are placed using USDC, or ATHL tokens from the user’s wallet.
 - Validate sufficient token balance before placing a bet; display an error if insufficient.
 - Lock bet tokens in a smart contract or escrow until settlement.

Team Leaderboard

- **Location:** Below the Team Competitions section on the Fan Dashboard.
- **UI Design:**
 - Display a table with the top 10 most successful teams.
 - Table columns:
 - **Rank:** Position (1–10).
 - **Team Name/ID:** Name or ID of the team (e.g., “PetersonSprintTeam”).

- **Wins:** Number of competitions won.
 - **Win Rate:** Percentage of competitions won (Wins / Total Competitions).
 - **Total Competitions:** Number of competitions participated in.
- Include a “View All” link to a separate page listing all teams in rank order (optional for future implementation).
- **Data Calculation:**
 - Query the backend for team competition history.
 - Rank teams by number of wins; use win rate as a tiebreaker.
 - Update the leaderboard after each competition concludes.
- **Interactivity:**
 - Clicking a team name links to the team profile and team token purchase page.

ATHLScan Database For Public Viewing

Transaction Data

1. The system shall record all follower token transactions (purchases, sales, and issuer-initiated transfers) in a centralized, off-chain ATHLScan database, hosted under the ATHLScan tab on the platform’s website.
2. Note: ATHLScan should likely **pull data from a primary database** rather than serve as the primary database for all transactions. This approach keeps ATHLScan lightweight, aligns with its role as a transparency tool, and ensures the core ATHL platform remains resilient and scalable
3. The system shall capture transaction data for each follower token trade, including:
 - a. Transaction ID (unique alphanumeric hash).
 - b. Timestamp (UTC, formatted as YYYY-MM-DD HH:MM:SS).
 - c. From ID (anonymous code or public issuer username for opted-in public trades/holdings, or ATHL which would be for initial sale).
 - d. To ID (anonymous code or public issuer username for opted-in public trades/holdings).
 - e. Token name/symbol (e.g., @ShaquilleONeal).
 - f. Amount of token sold (e.g., 10.5 tokens, with 2 decimal precision).
 - g. Transaction type (purchase, issuer transfer, reversal).
 - h. Payment method (USDC, USDT, SOL, credit card, follower token purchase credits).

- i. Total trading fee (1% of transaction value in USD).
4. The system shall exclude sensitive data (e.g., IP addresses, credit card details, user emails) from the public ATHLScan ledger to protect privacy.
5. The system shall display a transaction status (e.g., “Confirmed,” “Pending,” “Reversed”) to indicate processing state, with pending transactions updating to confirmed within 15 seconds.
6. The system shall store transaction data in the same internal transaction database used by other platform features (e.g., user accounts, issuer dashboards), ensuring consistency.

Database Editability

1. The system shall allow authorized administrators to edit the ATHLScan database to reverse transactions in cases of hacks, market manipulation, or black swan events, with criteria determined case-by-case by platform management.
2. The system shall create a new ledger entry for each reversal, tagged as “Reversal,” referencing the original transaction ID, and including a brief reason (e.g., “Reversed due to detected hack on 2025-05-18”).
3. The system shall notify all affected parties (e.g., buyer, seller, issuer) via email when a transaction is reversed, including the reversal reason and new ledger entry ID.
4. The system shall maintain an internal audit log of all database edits, including administrator identity, timestamp, justification, and affected transaction IDs, accessible only to authorized personnel with role-based access control.
5. The system shall visually highlight reversed transactions in the ATHLScan interface (e.g., red text or “Reversed” badge) for transparency.

User Anonymity and Issuer Publicity

1. The system shall represent all users in the ATHLScan ledger by anonymous codes (e.g., anon_38c57x9p2) to protect privacy, except for issuer trades or holdings opted as public.
2. The system shall allow issuers to designate specific trades or follower token holdings as public, displaying their verified username (e.g., “@ShaquilleONeal”) instead of an anonymous code in the ledger.
3. The system shall use an anonymous code for non-public issuer trades and holdings, consistent with other users.
4. The system shall provide issuers an account settings option to toggle publicity for individual trades or holdings during trade confirmation and portfolio management.

Multiple Anonymous USER Codes (We do not want trading data to be weaponized against our users, so while all trades are public, forensically reverse engineering user portfolio and trade behavior will be much more difficult.)

1. The system shall assign each user 5 initial anonymous codes upon account creation, with up to 30 codes for heavy traders, linked to their username and managed automatically.
2. The system shall randomly select one of the user's anonymous codes for each purchase recorded in the ledger, except for issuer-publicized trades.
3. The system shall select an anonymous user code for the available set for each purchase, used for that asset's ledger entries (purchase and eventual sale), treating it as a mini trading account.
4. The system shall automatically generate an additional anonymous user code for about every 200 trades, used randomly alongside existing codes for non-asset-specific entries.
5. The system shall ensure anonymous user codes are random, alphanumeric (8-12 characters), and publicly unlinked to personal data to prevent tracing.
6. The system shall securely map anonymous user codes to usernames in the internal database, accessible only to authorized personnel with role-based access control.

Trading Fees

1. The system shall apply a 1% trading fee to all token purchase transactions, calculated on the total transaction value in USD, regardless of payment method (stable coin, SOL, credit card, follower token purchase credits).
2. The system shall credit the issuer's share of fees to their account wallet, recorded in the internal database for dashboard access.

Issuer-Only Transfers (Issuers can give a small number of tokens to existing fans on using other platforms (Patreon, OF) as a bonus. This encourages those fans to join and support the issuer on the new ATHL platform.)

1. The system shall restrict the ability to make token transfers for issuers only, allowing them to transfer their own self purchased follower tokens to other users, recorded as "issuer transfer" in the ledger.
2. The system shall prohibit all other person-to-person transfers to maintain platform control.
3. The system shall allow issuers to input a recipient's username (e.g., provided via Patreon), resolving it internally to an anonymous code for ledger display (e.g., "Issuer_X -> anon_45678").
4. The system shall require MFA confirmation for issuer transfers exceeding 100 tokens per transfer to enhance security.

ATHLScan Interface

1. The system shall host the ATHLScan ledger on the platform's website under an "ATHLScan" tab, publicly accessible without login to maximize transparency.

2. The system shall update the ledger within 15 seconds of transaction confirmation for near real-time visibility.
3. The system shall provide a user-friendly interface with search and filter functions (e.g., by transaction ID, token symbol, anonymous user code, date range, transaction type).
4. The system shall display transactions in a paginated table, sorted by timestamp (newest first), with clickable transaction IDs linking to detailed views (e.g., all data fields).
5. The system shall include a “How to Read ATHLScan” FAQ section explaining anonymous codes, reversals, fees, and issuer publicity options.(TO BE DEVELOPED)
6. The system shall ensure the ATHLScan interface is responsive for desktop and mobile devices and complies with WCAG 2.1 accessibility standards (e.g., screen reader support, keyboard navigation).

Security and Scalability of Database

1. The system shall encrypt the ATHLScan data at rest and in transit to protect transaction data.
2. The system shall implement role-based access control, restricting database edits to authorized administrators with multi-factor authentication.
3. The system shall conduct regular security audits of the database and edit processes to detect vulnerabilities.
4. The system shall design the database to handle 1+ million daily transactions, using indexing (e.g., on transaction ID, timestamp) and caching for fast queries, with technology chosen by developers.
5. The ATHL primary database shall be backed up every 5 minutes using incremental backups, combined with transaction log backups for point-in-time recovery. Full backups shall be performed daily. Real-time replication to a secondary database shall be implemented for high availability. Backups shall be encrypted at rest and in transit, stored securely with 48 hour retention for incremental backups and 7-30 days for full backups, and regularly tested for restorability.

Compliance and User Support

1. The system shall comply with data protection regulations, prioritizing GDPR for Europe (secondary market) and CCPA for the USA (primary market), by anonymizing all personal data in the public ledger and obtaining consent for issuer publicity.
2. The system shall provide a privacy policy explaining how transaction data is stored, displayed, and edited in ATHLScan, accessible from the ATHLScan tab. TO BE DEVELOPED

ATHL Customer Support Request Page

Accessibility:

1. The system shall provide a customer support request page accessible via a clearly labeled link (“Support”) from the ATHLScan tab and each page in the ATHL platform.
2. The page shall be available to both authenticated users and non-authenticated users (ATHL Home page, ATHLScan), with authenticated users having pre-filled account details (e.g., username, email).

Support Form Functionality:

3. The system shall present a web-based support form with the following fields:
 - a. **Category:** A dropdown menu for issue type (e.g., Transaction Error, Fraudulent Activity, Account Issue, Login Issue, Platform Bug, General Inquiry, Other).
 - b. **Subject:** A text field for a brief summary of the issue (max 100 characters, required).
 - c. **Description:** A text area for detailed issue explanation (max 2000 characters, required).
 - d. **Transaction ID:** An optional field for entering a transaction ID (auto-populated if the user accesses the form from a specific transaction in ATHLScan).
 - e. **Attachments:** An optional file upload field for screenshots or documents (max 5 files, 10MB total, supporting formats: PNG, JPEG, PDF).
 - f. **Contact Email:** A required field for non-authenticated users; pre-filled and read-only for authenticated users.
 - g. **Priority Level:** A dropdown for urgency (e.g., Low, Medium, High), defaulting to Medium, with guidance on when to select High (e.g., suspected fraud).
4. The form shall include client-side validation to ensure required fields are completed and inputs meet format requirements (e.g., valid email, file size limits).

User Experience:

5. The system shall display clear instructions on the support page, explaining how to fill out the form and what information to include for faster resolution.

6. The system shall provide a confirmation message upon successful form submission, including a support ticket ID for tracking.

Integration with ATHLScan:

7. The system shall enable users to initiate a support request directly from a specific transaction in ATHLScan by clicking a "Report Issue" link, which pre-populates the Transaction ID and links the request to the transaction's details.
8. The system shall allow ATHLScan to display the status of support requests related to specific transactions (e.g., "Pending," "Resolved") for authenticated users. This list would show up on the support page, just below the entry fields for a new support request.

Security and Privacy:

9. The system shall encrypt all support request data (including attachments) at rest and in transit to protect sensitive user information.
10. The system shall implement rate-limiting and CAPTCHA (e.g., reCAPTCHA) to prevent spam or abuse of the support form.
11. The system shall restrict access to submitted support requests to authorized support staff, with role-based access controls (RBAC).
12. The system shall anonymize non-authenticated user data (e.g., IP addresses) in compliance with applicable privacy regulations (e.g., GDPR, CCPA).

Ticket Management and Notifications:

13. The system shall generate a unique support ticket ID for each submission and store the request in a customer support database.
14. The system shall send an automated email confirmation to the user's provided email address, including the ticket ID, summary, and estimated response time.
15. The system shall notify users via email or in-platform notifications (for authenticated users) when the ticket status updates (e.g., "In Progress," "Resolved").
16. The system shall allow support staff to categorize, prioritize, and assign tickets based on form inputs (e.g., Category, Priority Level).

Tracking and Follow-Up:

17. The system shall provide authenticated users with a “My Support Tickets” section (accessible via their account dashboard) to view the status, history, and details of their support requests.
18. The system shall allow users to reply to ongoing tickets via the support page or email to provide additional information or clarification.
19. The system shall enable support staff to request further details from users through the ticket system, with notifications sent to the user.

Performance and Scalability:

20. The system shall handle up to 1,000 support requests per hour without performance degradation, assuming peak trading activity.
21. The system shall ensure form submission completes within 3 seconds under normal conditions.

Analytics and Reporting:

22. The system shall log support request metadata (e.g., category, priority, submission time, resolution time) for internal analytics.
23. The system shall generate reports for support staff to identify common issues (e.g., frequent transaction errors) and improve platform functionality.

Compliance and Auditability:

24. The system shall retain support request data for at least 12 months (or as required by applicable regulations) for audit purposes.
25. The system shall log all staff interactions with support tickets (e.g., views, updates) for accountability.

Requirements to Foster an Active Trading Environment

1. Purchase Limits / anti hording:

- a. The system shall impose a per-issuer purchase limit for token sales. This will be set by the issuer and the options are: 10%, 1%, 0.1% and 0.01% of total tokens. Currently there is a limit of 10,000 tokens or 1% as the default on the website
- b. The system shall allow ATHL administrators to adjust purchase limits dynamically based on issuer popularity or trading activity, with limits communicated transparently on the trading page.

2. Tiered Pricing Enforcement:

- a. The system shall enforce the linear pricing model starting at \$1 and going up to as low as \$4 to \$24 for initial token sales. This will result in fundraising increments of \$2 million to \$10 million for this issuer, and gross proceeds to ATHL of \$2.5 million to \$12.5 million.
- b. The system shall prioritize smaller retail purchases (e.g., under 100 tokens) in lower price tranches to favor fans over large investors.

3. Liquidity Incentives:

- a. The system shall offer trading incentives to encourage token circulation, such as reduced trading fees (e.g., 0.5% instead of 1%) for users who trade at least 10 tokens weekly.
- b. The system shall reward traders in ATHL who add liquidity to token markets. We will follow a similar reward system to this:
<https://www.dydx.foundation/blog/liquidity-provider-rewards>

4. Secondary Market Controls:

- a. No single person can sell more than 5,000 tokens within 24 hours to prevent large holders from dumping tokens and crashing prices. This amount can be adjusted by admins at any time.
- b. The system shall introduce a price stabilization mechanism, such as a 24-hour price floor where sales are not allowed below 85% of the previous day's average price), to reduce volatility and maintain fan confidence.

5. Integration with Customer Support:

- a. The system shall allow users to report suspected market manipulation or unfair token distribution via the customer support request page with a dedicated "Market Abuse" category prioritized for review.
- b. The system shall enable ATHL staff to freeze suspicious accounts (e.g., linked to excessive token holdings) pending investigation, with clear communication to affected users.

6. Security and Fairness:

- a. The system shall encrypt all trading data at rest and in transit, aligning with ATHLScan's security requirements, to protect against unauthorized access or manipulation.

Security Requirements for ATHL Follower Token Trading Platform

Functional Requirements

Purchase Anomaly Detection and CAPTCHA Verification

Description: The system shall monitor purchase API calls for anomalies (e.g., rapid buys or coordinated purchases across multiple tokens) and implement a risk-based CAPTCHA

(e.g., reCAPTCHA v3 or hCaptcha) before purchase order confirmation to deter bots and snipers during the 30-day purchase-only phase.

Threats Addressed: Sniping (exploiting launch pricing), bot usage (mass purchases), market manipulation (pump-and-dump schemes).

Detection Criteria:

- Purchases exceeding 10 in 1 minute across different tokens from a single account.
- Multiple accounts from the same IP address purchasing simultaneously.
- Failed or suspicious CAPTCHA responses (e.g., reCAPTCHA v3 scores <0.5, hCaptcha failures).
- Purchase attempts bypassing CAPTCHA or using automated solvers (e.g., completion in <2 seconds).

Implementation Details:

- Analyze purchase API logs for frequency, volume, and IP patterns using a lightweight machine learning model (e.g., isolation forest in scikit-learn) to flag outliers.
- Integrate Alchemy wallet device fingerprints (e.g., device ID, browser agent) to detect shared devices.
- Deploy reCAPTCHA v3 for low-friction background scoring, escalating to hCaptcha image/audio challenges for high-risk actions (e.g., >5 buys/minute, new IPs, or unverified devices).
- Validate CAPTCHA responses server-side, flagging anomalies (e.g., completions <2 seconds).
- Monitor purchases against the ATHL pricing curve (\$1–\$24, 1M steps) to detect buys timed with price jumps.
- Host anomaly detection and CAPTCHA logic on a cloud server (e.g., AWS EC2) with real-time processing.
- Notify users of CAPTCHA requirements via in-platform alerts (e.g., “Please verify to confirm purchase”)

Metrics:

- Detection rate: >95% for sniping and bot-driven purchases.
- Latency: <30ms for anomaly scoring, <100ms for CAPTCHA validation.
- False positive rate: <0.3% to minimize legitimate user disruption.
- CAPTCHA success rate: >98% for legitimate users.

Priority Rationale:

- **Cost:** Low (uses existing API data, open-source ML, free reCAPTCHA v3 up to 1M requests/month, hCaptcha at ~\$0.01–\$0.05 per 1,000 checks).
- **Impact:** High (prevents sniper-driven losses, deters bots, protects pricing curve).

- **Complexity:** Low (simple API and CAPTCHA integration).
Justification: Critical for protecting token launches, leverages existing infrastructure, and adds a low-cost, high-impact CAPTCHA layer to combat automated purchases with minimal user friction.

Randomized Purchase Execution Delays

Description: The system shall apply randomized micro-delays (50–100ms) to purchase execution during token launches to disrupt bot-driven latency exploitation and deter sniping.

Threats Addressed: Sniping (API latency exploitation).

Detection Criteria:

- Purchase attempts within 100ms of a token launch or pricing curve update.
- High-frequency API calls (>10 requests/second) from single accounts.

Implementation Details:

- Monitor purchase API timestamps to identify sub-100ms attempts.
- Introduce randomized delays using a server-side script (e.g., Python with `random.uniform`) in the purchase execution engine.
- Configure delays via a platform admin dashboard, active for the first 24 hours of launches

Metrics:

- Sniping reduction: >90% decrease in sub-100ms purchases.
- User disruption: <0.03% user complaints about delays.

Priority Rationale:

- **Cost:** Low (minor code change in purchase engine).
- **Impact:** High (disrupts sniper bots exploiting low initial prices).
- **Complexity:** Low (minimal update to transaction processing).
Justification: Simple, immediate anti-sniping measure with no external dependencies, critical for fair pricing during launches.

User Risk Score Calculation

Description: The system shall calculate a real-time user risk score (0–100) based on behavioral, transactional, and contextual signals to identify potential threats during purchases or logins.

Threats Addressed: Sniping, bot usage, scamming, market manipulation (high-risk accounts).

Detection Criteria:

- **Behavioral:** Multiple logins from different IPs (>3 in 24 hours), rapid clicks (<100ms intervals), inconsistent device fingerprints (e.g., changing browser agents).
- **Transactional:** High purchase frequency (>10 buys/hour), purchases nearing the 0.1% holding limit for famous tokens, buys timed with external hype (e.g., X posts).
- **Contextual:** Unverified social sign-on attempts, X posts linking the user to scams, disposable email domains (e.g., tempmail.com).

Implementation Details:

- Collect login and device data via Alchemy wallet APIs (e.g., IP address, device ID).
- Monitor purchase API calls for frequency and volume against the ATHL pricing curve.
- Use Grok's API to analyze X posts for user mentions or scam keywords (e.g., "pump," "scam").
- Deploy a logistic regression model (e.g., scikit-learn) with weighted features (40% transactional, 30% behavioral, 30% contextual), normalizing scores to 0–100.
- Host the risk scoring engine on a cloud platform (e.g., AWS Lambda) with event-driven processing (e.g., Apache Kafka).

Metrics:

- Accuracy: >90% correlation with confirmed threats.
- Latency: <50ms for real-time scoring.
- False positive rate: <0.3% to avoid disrupting legitimate users.

Priority Rationale:

- **Cost:** Low (uses existing Alchemy data, open-source ML, minimal Grok API usage).
- **Impact:** High (enables targeted interventions like KYC).
- **Complexity:** Moderate (requires feature engineering and data pipeline setup).

Justification: Essential for prioritizing high-risk accounts, leverages available data, and supports scalable threat mitigation.

User Risk Score Triggers

Description: The system shall trigger actions based on user risk score tiers within 5 minutes of detection: low (0–30, no action), medium (31–70, enhanced monitoring), high (71–100, require KYC via a provider like Persona or restrict purchases to \$100/day until completed).

Threats Addressed: Sniping, bot usage, scamming, market manipulation (anonymous or high-risk accounts).

Detection Criteria:

- Risk scores >70, especially tied to rapid purchases (>10 buys/hour) or suspicious IPs (e.g., VPNs, known botnets).
- Accounts with high scores attempting purchases near the 0.1% holding limit.

Implementation Details:

- Implement rule-based triggers in the risk scoring engine (e.g., “if score >70, initiate KYC”).
- Integrate with a KYC provider API (e.g., Persona) for ID verification (e.g., driver’s license scan).
- Monitor purchase API for restricted accounts attempting to bypass limits.
- Notify users via email and in-platform alerts about KYC requirements or restrictions.
- Use manual checks as a fallback for low volumes.

Metrics:

- Intervention success: >90% mitigation of identified threats.
- Verification latency: <5 seconds for KYC API calls.
- User drop-off: <5% due to KYC friction

Priority Rationale:

- **Cost:** Moderate (KYC fees, ~\$0.50–\$2 per check, applied selectively).
- **Impact:** High (deters anonymous threats, complements email verification).
- **Complexity:** Moderate (requires API integration and user communication).

Justification: Strong deterrent for anonymous threats, balances cost by targeting high-risk accounts, and aligns with compliance needs.

Non-X Social Media Monitoring

Description: The system shall monitor Telegram, Discord, and Reddit for scam and manipulation signals using open-source NLP and web scraping tools, flagging high-risk posts for human review and correlating with purchase activity within 5 minutes.

Threats Addressed: Scamming (phishing, fake promotions), market manipulation (coordinated hype).

Detection Criteria:

- Phishing links or fake token promotions in Telegram/Discord (e.g., “Buy now at [link]”).
- Coordinated hype on Reddit (e.g., posts with low karma but >50 upvotes).
- Sudden member surges in Telegram groups (>100 joins/day) or spam-like messages.

Implementation Details:

- Use Telegram's Bot API and Discord's Webhook API to stream messages in real time.
- Scrape Reddit's r/cryptocurrency and token-specific subreddits hourly using Scrapy or BeautifulSoup.
- Apply Hugging Face's BERT model for sentiment analysis (e.g., detect excessive positivity) and keyword detection (e.g., "scam," "pump").
- Flag posts with high engagement (>50 reactions) or from new accounts (<30 days old).
- Correlate flagged posts with purchase spikes (>5% volume increase in 10 minutes) using a rule engine.
- Deploy a data pipeline (e.g., Apache Airflow) on a cloud server (e.g., AWS EC2), integrating NLP outputs with the risk scoring engine.
- Route flagged posts to a moderator dashboard for review.

Metrics:

- Detection accuracy: >80% for scam/manipulation signals.
- Latency: <5 minutes from post to flag.
- False positive rate: <10% to reduce moderator workload.

Priority Rationale:

- **Cost:** Low (open-source tools, minimal API fees for Telegram/Discord).
- **Impact:** High (broadens scam detection beyond X).
- **Complexity:** Moderate (requires NLP setup and pipeline integration).

Justification: Complements X monitoring, high-impact for scams, and cost-effective with open-source tools.

Grok-Based X Social Media Monitoring

Description: The system shall use Grok's API to monitor X posts in real time for scam and manipulation signals (e.g., pump-and-dump keywords, issuer handle mentions) and correlate with purchase spikes within 1 minute.

Threats Addressed: Scamming (fake promotions), market manipulation (hype-driven pumps).

Detection Criteria:

- Posts with >100 likes or retweets mentioning tokens or issuers' verified handles.
- Coordinated posts (e.g., similar text from multiple accounts within 10 minutes).
- Sudden spikes in token mentions alongside purchase surges (>5% volume increase).

Implementation Details:

- Configure Grok's API to scan posts mentioning tokens or issuers' handles, using keyword filters (e.g., "buy," "moon") and sentiment analysis.
- Analyze post engagement metrics (likes, retweets) and posting patterns (e.g., cadence, account age).
- Correlate post timestamps with purchase API data using a rule engine (e.g., "flag if mentions increase 50% and purchases rise 5%").
- Integrate Grok's API with the risk scoring engine via a cloud-based pipeline (e.g., AWS Lambda).
- Add human review for flagged posts via a moderator dashboard.

Metrics:

- Signal accuracy: >85% correlation with purchase spikes.
- Detection latency: <1 minute from post to flag.
- False positive rate: <5% to minimize manual review.

Priority Rationale:

- **Cost:** Moderate (Grok API fees, ~\$0.01–\$0.05 per query).
- **Impact:** High (catches external manipulation).
- **Complexity:** Low (leverages Grok's X integration).
- **Justification:** High-impact for detecting scams, straightforward to implement, though API costs slightly lower priority.

Strong Email Verification for User Accounts

Description: The system shall implement strong email verification for all users during account creation to ensure email addresses are valid, active, and not associated with disposable or fraudulent services, reducing bot-driven account creation.

Threats Addressed: Bot usage (mass account creation), scamming (fake accounts), market manipulation (pump-and-dump via disposable emails).

Detection Criteria:

- Invalid email syntax or non-existent domains (e.g., missing MX records).
- Disposable email services (e.g., Temp-Mail, Guerrilla Mail).
- High-risk emails linked to spam, phishing, or bot activity (e.g., via reputation scoring).
- Newly created or inactive email accounts (<30 days old).

Implementation Details:

- Integrate a third-party email verification service (e.g., ZeroBounce, NeverBounce) to check syntax, domain validity, disposable status, and reputation in real-time during signup.

- Reject disposable or high-risk emails (e.g., reputation score <50/100) and prompt users to provide a valid email.
- Send a confirmation link or OTP to verify email ownership, requiring user action to complete signup.
- Store verification results in the user database (e.g., AWS RDS) for audit trails.
- Host verification logic on a cloud server (e.g., AWS Lambda) with API integration

Metrics:

- Bot account reduction: >90% decrease in disposable email signups.
- Verification latency: <1 second for email checks.
- False rejection rate: <0.5% for legitimate emails.

Priority Rationale:

- **Cost:** Low (~\$0.001–\$0.01 per email, \$10–\$100 for 10,000 users).
- **Impact:** High (blocks bot accounts, supports secure email OTPs).
- **Complexity:** Low (standard API integration).

Justification: Critical for preventing bot-driven pump-and-dump schemes, cost-effective, and enhances MFA security by ensuring trusted email addresses.

Multi-Factor Authentication (MFA) with Flexible Options

Description: The system shall require multi-factor authentication (MFA) for all users during login and high-risk actions (e.g., purchases, withdrawals), offering biometrics, passkeys, authenticator apps, and email OTPs to prevent unauthorized access and bot activity.

Threats Addressed: Bot usage (automated logins), scamming (account takeovers), market manipulation (hijacked accounts for pump-and-dump).

Detection Criteria:

- Logging in.
- High-risk actions (e.g., >5 buys/hour) from accounts without MFA enabled.
- Suspicious passkey or biometric failures (e.g., repeated denials).

Implementation Details:

- Require MFA setup during account creation, offering:
 - **Biometrics:** Device-based fingerprint or facial recognition (e.g., Apple Face ID, Android BiometricPrompt).
 - **Passkeys:** Passwordless authentication via WebAuthn/FIDO2, synced via platform APIs (e.g., Apple, Google).
 - **Authenticator Apps:** Time-based OTPs (e.g., Google Authenticator, Microsoft Authenticator).

- **Email OTPs:** One-time codes sent to verified email addresses.
- Integrate biometric and passkey APIs into the authentication flow, with authenticator apps and email OTPs as defaults.
- Use verified emails (from strong email verification) for OTP delivery, leveraging low-cost providers (e.g., AWS SES, ~\$0.0001–\$0.001 per email).
- Enforce MFA for logins and actions exceeding risk thresholds (e.g., purchases >\$100 or >5 buys/hour).
- Provide user guides (e.g., in-platform videos) to simplify MFA setup.
- Store MFA configurations securely in the user database (e.g., AWS RDS with encryption).
- Host authentication logic on a cloud server (e.g., AWS EC2) with real-time processing.

Metrics:

- MFA adoption rate: >95% of active users.
- Unauthorized access prevention: >98% reduction in account takeovers.
- Bot login reduction: >90% decrease in automated login attempts.
- Latency: <100ms for MFA validation

Priority Rationale:

- **Cost:** Low (free for biometrics/authenticator apps, ~\$0.0001–\$0.001 per email OTP, passkey APIs included in platform SDKs).
- **Impact:** High (prevents takeovers, deters bots, protects trading integrity).
- **Complexity:** Moderate (requires API integration and user education).
- **Justification:** Essential for securing accounts and reducing bot-driven manipulation, balances accessibility with robust security, and avoids SMS costs.

Behavioral Biometrics for Bot Detection

Description: The system shall analyze user interactions (e.g., mouse movements, typing patterns) via Alchemy wallet sign-on to detect bot-driven purchases in real time.

Threats Addressed: Bot usage (automated purchases), scamming (hijacked accounts).

Detection Criteria:

- Rapid, repetitive clicks (<100ms intervals) or direct API calls bypassing UI.
- Inconsistent device fingerprints (e.g., multiple IPs or browser agents in one session).
- Non-human navigation patterns (e.g., no mouse movements, straight-to-purchase actions).

Implementation Details:

- Integrate an open-source behavioral biometrics SDK (e.g., FingerprintJS Pro) with Alchemy's sign-on flow.
- Use anomaly detection (e.g., clustering algorithm) to flag non-human behavior based on interaction telemetry.
- Cross-reference with purchase API logs for correlated activity (e.g., rapid buys).
- Deploy the SDK within the authentication layer, processing telemetry on a cloud server (e.g., AWS EC2).
- Feed anomalies into the risk scoring engine for KYC triggers.

Metrics:

- Bot detection rate: >92% for automated purchases.
- Latency: <50ms for analysis.
- User disruption: <0.03% impact on legitimate users.

Priority Rationale:

- **Cost:** Moderate (SDK licensing, ~\$0.10–\$0.50 per user/month, or open-source alternatives).
- **Impact:** High (catches sophisticated bots).
- **Complexity:** Moderate (requires SDK integration and telemetry pipeline).
- **Justification:** Effective for detecting advanced bots, though moderate costs and integration effort adjust its priority.

User Risk Score Transparency

Description: The system shall provide users with anonymized risk score feedback (e.g., “Your account is low risk”) via in-platform notifications or email to build trust and explain restrictions.

Threats Addressed: Scamming, market manipulation (user trust and awareness).

Detection Criteria:

- User confusion or support tickets about purchase restrictions or KYC requests.
- Low user engagement with notifications (<50% read rate).

Implementation Details:

- Integrate risk score feedback into the notification system, triggered by score changes or interventions.
- Display messages in the platform UI and send emails for high-risk actions.
- Monitor support tickets via a customer service platform (e.g., Zendesk) for restriction-related complaints.
- Track notification read rates using analytics (e.g., Mixpanel).
- Survey users for satisfaction with transparency (e.g., post-notification feedback)

Metrics:

- Transparency score: >4.7/5 user satisfaction in surveys.
- Notification delivery: <10 seconds from trigger to delivery.

Priority Rationale:

- **Cost:** Low (uses existing notification infrastructure).
- **Impact:** Moderate (enhances user trust, supports trading goals).
- **Complexity:** Low (simple UI and email integration).

Justification: Low-cost trust builder, indirectly supports threat mitigation by improving user cooperation.

Monitor Trading Patterns for Manipulation with Staff Alerts

Description: The system shall monitor trading patterns for signs of manipulation, including coordinated bulk purchases or sales, spoofing, and other account activity coordination, using automated algorithms, and send alerts to ATHL staff for review within 5 minutes of detection to prevent market manipulation and ensure a fair trading environment.

Threats Addressed: Market manipulation (coordinated bulk purchases/sales, spoofing, account activity coordination), bot usage (automated manipulation).

Detection Criteria:

- **Coordinated Bulk Purchases or Sales:** Multiple accounts (e.g., >3 accounts) executing large buy or sell orders (>0.05% of token supply) within a short timeframe (e.g., <10 minutes), especially if tied to price movements (>5% change in 10 minutes).
- **Spoofing:** High-frequency order placements and cancellations (e.g., >10 orders placed and canceled within 1 minute) without execution, correlated with price fluctuations (>2% change in 5 minutes).
- **Account Activity Coordination:** Synchronized trading patterns across accounts, such as similar buy/sell timings (<1 minute apart), shared IPs, or device fingerprints (e.g., via Alchemy wallet data), indicating potential collusion.
- **Suspicious Patterns:** Trades timed with external hype (e.g., social media spikes detected via Grok's API), rapid buy-sell cycles (<5 minutes apart) to artificially inflate volume, or layering (placing orders on both sides of the order book to create false depth)

Implementation Details:

- Collect trading data (buy/sell orders, cancellations, timestamps) from the purchase and sell APIs, storing them in a centralized database (e.g., AWS RDS) for real-time analysis.
- Use Alchemy/Privy wallet APIs to gather device fingerprints and IP addresses for cross-account correlation.
- Deploy automated algorithms to detect manipulation patterns:
 - **Clustering Algorithm:** Group accounts by trading behavior (e.g., k-means clustering in scikit-learn) to identify synchronized patterns (e.g., similar buy/sell timings or volumes).
 - **Time-Series Analysis:** Monitor order book activity for spoofing (e.g., high-frequency order placement/cancellation) using a sliding window approach (e.g., 1-minute windows) to detect rapid cancellations.
 - **Rule-Based Engine:** Flag coordinated activity (e.g., >3 accounts with shared IPs trading within 10 minutes) and correlate with price/volume spikes (>5% change).
- Integrate with existing social media monitoring (Grok-Based X and Non-X) to correlate trading patterns with external hype signals (e.g., sudden token mentions).
- Generate alerts for ATHL staff via a moderator dashboard (e.g., same dashboard used for social media monitoring) and email notifications, including details like account IDs, timestamps, trading volumes, and suspected manipulation type (e.g., "Spoofing detected: 15 orders placed/canceled in 1 minute").
- Host the monitoring system on a cloud server (e.g., AWS EC2) with real-time processing, leveraging event-driven architecture (e.g., Apache Kafka) for scalability.
- Provide staff with options to take action (e.g., suspend accounts, pause trading, or initiate KYC) directly from the dashboard.

Metrics:

- Detection accuracy: >90% for confirmed manipulation patterns.
- Alert latency: <5 minutes from detection to staff notification.
- False positive rate: <5% to minimize staff workload.
- Staff response time: <30 minutes for review and action on high-priority alerts.

Priority Rationale:

- **Cost:** Moderate (uses existing trading data and social media pipelines, open-source ML libraries like scikit-learn, minor dashboard updates).

- **Impact:** High (prevents pump-and-dump, spoofing, and other manipulations, ensuring market fairness).
- **Complexity:** Moderate (requires new algorithms for spoofing and coordination, integration with existing systems).
- **Justification:** Critical for maintaining a fair trading environment, directly addresses coordinated manipulation and spoofing not fully covered by existing requirements, and leverages existing infrastructure for cost efficiency. Staff alerts ensure timely human oversight for complex threats.

Dynamic Trading Pause Triggers

Description: The system shall adjust trading pause thresholds (e.g., 20% price dip in 24 hours) after the 30-day purchase-only phase, based on token age or volatility, pausing trading for 1–4 hours with user notifications to prevent crash exploitation.

Threats Addressed: Market manipulation (crash exploitation, post-purchase phase).

Detection Criteria:

- Price dips >20% after selling begins, tied to large sell orders (>0.05% of token supply).
- Rapid sell-off patterns post-hold period expiration.

Implementation Details:

- Monitor price feeds (available post-purchase phase) using a price tracking API.
- Analyze sell API logs for volume and timing once selling is enabled.
- Use a machine learning model (e.g., regression) to predict optimal pause thresholds based on historical price data.
- Integrate pause logic with price monitoring and sell API post-30 days.
- Automate notifications via email and in-platform alerts.
- Deploy on a cloud server with real-time price processing.

Metrics:

- Pause effectiveness: >90% reduction in crash-driven losses.
- User disruption: <0.05% impact on legitimate traders.

Priority Rationale:

- **Cost:** Low (uses price data, existing notification system).
- **Impact:** High (prevents post-purchase crashes).
- **Complexity:** Moderate (requires price data and AI tuning, unavailable in purchase phase).

Justification: De-prioritized due to purchase-only focus, relevant only after selling begins.

Non-Functional Requirements

Real-Time Processing

Description: The system shall process purchase API calls, Alchemy wallet telemetry, and social media data (X and non-X) with <30ms latency for risk scoring and <100ms for interventions to ensure real-time threat detection.

Detection Criteria:

- Processing bottlenecks in purchase API or social media pipelines (e.g., >50ms delays).
- High latency in risk score calculations or KYC triggers.

Implementation Details:

- Use monitoring tools (e.g., Prometheus, Grafana) to track API response times and pipeline latency.
- Conduct load tests with simulated purchases to identify bottlenecks.
- Deploy the system on a cloud-based, event-driven architecture (e.g., AWS Lambda, Apache Kafka) with low-latency processing.
- Optimize database queries for real-time access.

Metrics:

- Throughput: 5,000 transactions/second during launches.
- Latency: <30ms for 95% of risk scoring events.

Priority Rationale: Critical for real-time sniping and bot detection, leverages cloud infrastructure for low complexity.

Scalability

Description: The system shall scale to handle 10,000 transactions/second during token launches, with <3-minute node addition time to accommodate peak purchase volumes.

Detection Criteria:

- Performance degradation during high-volume simulations (>5,000 tx/s).
- Slow node scaling (>5 minutes) under load.

Implementation Details:

- Conduct stress tests with synthetic purchase data using tools like Locust.
- Monitor cloud auto-scaling metrics (e.g., AWS Auto Scaling group events).

- Use a cloud-native architecture (e.g., Kubernetes on AWS EKS) with auto-scaling enabled.
- Pre-provision resources for anticipated launch volumes.

Metrics:

- Peak load capacity: 10,000 tx/s.
- Resource utilization: <75% CPU/memory during peaks.

Priority Rationale: Ensures platform reliability during launches, critical for user retention and day trading.

Compliance

Description: The system shall maintain 100% audit trails for all purchases, issuer ID checks, email verifications, and on-demand KYC, complying with AML and GDPR regulations.

Detection Criteria:

- Missing audit logs for purchases or KYC actions.
- Incomplete issuer ID records for famous issuers.

Implementation Details:

- Implement logging middleware (e.g., AWS CloudTrail) to capture all system actions.
- Build an audit dashboard to verify log completeness.
- Store audit logs in a secure database (e.g., AWS RDS with encryption).
- Generate compliance reports for regulators via automated scripts.

Metrics:

- Audit trail completeness: 100% for all transactions.
- Incident reporting time: <12 hours for scams or manipulation.

Priority Rationale: Mandatory for legal compliance, leverages existing verification processes for low cost.

User Experience

Description: The system shall minimize disruption to legitimate users (<0.03% impacted by false positives) and provide transparent explanations for interventions (>4.7/5 user satisfaction) to support an active trading environment.

Detection Criteria:

- User complaints about purchase restrictions or delays (>10 tickets/day).
- Low satisfaction scores in post-intervention surveys (<4/5).

Implementation Details:

- Monitor support tickets via a customer service platform (e.g., Zendesk).
- Collect user feedback through in-platform surveys after interventions.
- Integrate transparent notifications (e.g., “Purchase restricted due to high activity”) into the platform UI and email system.
- Resolve complaints within 30 minutes via a support team.

Metrics:

- Disruption rate: <0.03% of legitimate users impacted.
- Resolution time: <30 minutes for user complaints.

Priority Rationale: Balances security with user retention, critical for day trading, and uses existing systems for low complexity.

Adaptability

Description: The system shall update risk scoring models and social media monitoring rules every 6 hours and adapt to new threats (e.g., novel bot patterns) within 20 minutes to ensure ongoing effectiveness.

Detection Criteria:

- Declining detection accuracy (<80%) for new scam tactics.
- Delayed model updates (>12 hours) for emerging threats.

Implementation Details:

- Track model performance metrics (e.g., precision, recall) using a monitoring dashboard.
- Log new threat patterns via moderator feedback and social media flags.
- Deploy an online learning pipeline (e.g., TensorFlow with incremental updates) for risk models.
- Automate rule updates for social media monitoring via a configuration management tool (e.g., Ansible).

Metrics:

- Model update frequency: Every 6 hours.
- Threat adaptation time: <20 minutes.

Priority Rationale: Ensures system evolution, moderate complexity with cloud-based ML pipelines.

Updated ATHL Security Requirements: Polymarket-Related Additions

Follower Authenticity Checks for Issuers

Description: The system shall check follower authenticity for issuers with >10,000 followers using a third-party API (e.g., HypeAuditor), flagging accounts with >20% fake followers, and display an authenticity badge (e.g., "<10% Bots") on token purchase pages.

Threats Addressed: Social media-driven manipulation, issuer fraud (fake follower inflation).

Detection Criteria:

- Follower bot percentage >20% (e.g., low engagement, generic profiles).
- Suspicious follower growth (e.g., >50% increase in 30 days).
- API errors or unverifiable accounts.

Implementation Details:

- Integrate HypeAuditor API to analyze follower authenticity for issuers with >10,000 followers during onboarding and monthly thereafter.
- Store authenticity results (e.g., bot percentage) in the user database (AWS RDS, encrypted).
- Display badges on token pages (e.g., "Verified <10% Bots," "Warning: >20% Bots") within 5 seconds of API response.
- Flag high-bot accounts (>20%) for manual review via admin dashboard.
- Host API calls on a cloud server (e.g., AWS Lambda) with retry logic for failures.

Metrics:

- Detection accuracy: >90% for fake followers.
- Latency: <5 seconds for API checks.
- False positive rate: <5% for legitimate accounts.

Priority Rationale:

- **Cost:** Low (~\$0.10 per check, ~\$100 for 1,000 issuers).

- **Impact:** High (enhances trust, deters manipulation).
- **Complexity:** Low (API integration).
- **Justification:** Polymarket doesn't rely on social influence, but ATHL's token value ties to issuer popularity, necessitating follower authenticity checks to prevent deceptive hype, unlike Polymarket's oracle-based security.

3. Issuer Multi-Factor Authentication (MFA)

Description: The system shall mandate multi-factor authentication (MFA) for all issuers during login and high-risk actions (e.g., token transfers, publicity toggles), supporting authenticator apps and email OTPs, to prevent account hijacking.

Threats Addressed: Issuer fraud (account takeovers), social media-driven manipulation (hijacked promotions).

Implementation Details:

- Integrate Google Authenticator and AWS SES for email OTPs, enforcing MFA setup during issuer onboarding.
- Require MFA for logins, token transfers (>100 tokens), and publicity setting changes.
- Store MFA configurations in the user database (AWS RDS, encrypted) with RBAC access.
- Notify issuers of failed MFA attempts via email within 5 minutes.
- Host authentication logic on a cloud server (e.g., AWS EC2) with real-time processing.

Metrics:

- MFA adoption rate: >95% of issuers.
- Unauthorized access prevention: >98% reduction in takeovers.
- Latency: <100ms for MFA validation.

Priority Rationale:

- **Cost:** Low (~\$0.0001 per OTP, free for authenticator apps).
- **Impact:** High (secures issuer accounts, protects promotions).

- **Complexity:** Moderate (API integration, user education).
- **Justification:** Polymarket's wallet-based security doesn't address issuer account risks, but ATHL's social-driven model requires MFA to prevent hijacked accounts from fueling fraud or manipulation.

4. Trade Reversal Framework

Description: The system shall implement a trade reversal framework, defining and publishing criteria for reversals (e.g., bot activity, manipulation), logging reversals on ATHLScan with reasons, and notifying affected users within 5 minutes to ensure transparency.

Threats Addressed: Trade reversal disputes, bot exploitation, crowd manipulation.

Detection Criteria:

- Trades exceeding 2 buys/second or <100ms click intervals (bot activity).
- User complaints or support tickets about reversals (>5 per hour).
- More criteria to be developed

Implementation Details:

- Define reversal criteria in a rule engine (e.g., "flag >3 buys/second, correlate with Grok API social spikes").
- Log reversals in ATHLScan as new entries tagged "Reversal," referencing original transaction ID and reason (e.g., "Reversed: Bot activity on 2025-05-22").
- Notify affected users (buyer, seller, issuer) via email (AWS SES) and in-platform alerts within 5 minutes, including reversal details.
- Store reversal logs in the internal database (AWS RDS, encrypted) with RBAC access.
- Update ATHLScan UI to highlight reversals (e.g., red text, "Reversed" badge).
- Host rule engine and notification logic on a cloud server (e.g., AWS Lambda).

Metrics:

- Notification delivery: >95% within 5 minutes.

- Dispute rate: <5% of reversed trades.
- Transparency score: >4.7/5 in user surveys.

Priority Rationale:

- **Cost:** Low (~\$0.0001 per email, \$0.01 per 1,000 UI updates).
- **Impact:** High (builds trust, reduces disputes).
- **Complexity:** Moderate (rule engine, UI integration).
- **Justification:** Unlike Polymarket's immutable trades, ATHL's reversal policy requires clear criteria and public logging to manage bot/manipulation risks and maintain user trust, critical for off-chain operations.

5. Cryptographic Trade Log Signing

Description: The system shall cryptographically sign all trade logs with HMAC-SHA256 to ensure integrity, storing them in the primary database, and audit daily against ATHLScan outputs to detect tampering.

Threats Addressed: Off-chain data integrity (tampering, insider threats).

Implementation Details:

- Implement HMAC-SHA256 signing for each trade log entry using Python's `hmac` module, storing signatures in AWS RDS (encrypted).
- Run daily cron jobs to audit logs against ATHLScan, flagging discrepancies for admin review via dashboard.
- Restrict database access with RBAC, requiring MFA for edits, and log all admin actions (timestamp, user ID, action).
- Host signing and audit logic on a cloud server (e.g., AWS EC2).

Metrics:

- Log integrity: 100% valid signatures.
- Audit discrepancies: <1% of trades.
- Admin access violations: <0.1% of attempts.

Priority Rationale:

- **Cost:** Low (~\$0.01 per 1,000 signatures, staff time for audits).
- **Impact:** High (ensures ATHLScan trust).
- **Complexity:** Low (standard cryptography, cron jobs).
- **Justification:** Polymarket's blockchain ensures immutability, but ATHL's off-chain ledger requires cryptographic signing to prevent tampering, critical for public transparency and regulatory audits.

6. ATHLScan Anonymization and Anti-Scraping

Description: The system shall enhance ATHLScan privacy by randomizing transaction IDs and user codes, rounding timestamps to 5 minutes with 1–2-minute batch updates, rate-limiting queries, and monitoring for exploitation to prevent deanonymization and scraping.

Threats Addressed: ATHLScan deanonymization, data scraping.

Implementation Details:

- Generate 12-character transaction IDs using UUID v4 (Python `uuid`) and 8–12-character user codes with `secrets` module, ensuring buy-sell matching per token.
- Round public timestamps to 5 minutes (e.g., “2025-05-22 12:35:00”), batch updates every 1–2 minutes, storing precise timestamps internally (AWS RDS).
- Implement rate-limiting with AWS WAF (<100 queries/hour per IP), triggering reCAPTCHA v3 for excess queries.
- Analyze access logs with AWS CloudWatch for bulk requests or pattern searches, alerting admins within 5 minutes.
- Store data in AWS RDS (encrypted), with RBAC and MFA for access.
- Update ATHLScan UI to reflect rounded timestamps and randomized IDs.

Metrics:

- Randomization strength: >99.9% unpredictability.
- Correlation reduction: >80% harder to link trades to events.

- Scraping prevention: >95% of attempts blocked.
- Detection accuracy: >90% for exploitation attempts.

Priority Rationale:

- **Cost:** Low (~\$0.01 per 1,000 IDs/timestamps, \$0.10 per 1,000 queries, \$0.50 per 1,000 log analyses).
- **Impact:** High (protects privacy, GDPR/CCPA compliance).
- **Complexity:** Moderate (API, log analysis setup).
- **Justification:** Polymarket's blockchain pseudonymity doesn't need active anti-scraping, but ATHLScan's public, off-chain data requires randomization, rate-limiting, and monitoring to prevent deanonymization, unlike Etherscan's wallet-based transparency.

7. Enhanced Bot and Sniper Detection

Description: The system shall enhance bot and sniper detection by flagging rapid trades (>3 buys/second, <100ms clicks), enforcing dynamic API limits (<2 requests/second during launches), and monitoring session continuity (>3 device/IP switches in 10 minutes) to trigger reversals.

Threats Addressed: Bot exploitation, sniper attacks, crowd manipulation.

Detection Criteria:

- Purchase API calls >2 buys/second or click intervals <100ms.
- API requests >2/second during token launches.
- Device/IP switches >3 in 10 minutes during high-risk actions (e.g., rapid buys).

Implementation Details:

- Update purchase API to flag >3 buys/second or <100ms clicks, using existing anomaly detection (e.g., scikit-learn isolation forest).
- Enforce dynamic API limits (<2 requests/second) during launches via server configuration (e.g., Nginx rate-limiting).

- Integrate FingerprintJS Pro (~\$0.10 per user/month) to track device/IP switches, flagging >3 in 10 minutes.
- Feed detections into the reversal framework, logging triggers in ATHLScan.
- Host detection logic on a cloud server (e.g., AWS EC2) with real-time processing.

Metrics:

- Bot detection rate: >92% for automated trades.
- Sniper reduction: >90% in rapid purchases.
- False positive rate: <0.3% for legitimate users.
- Latency: <50ms for detection.

Priority Rationale:

- **Cost:** Moderate (~\$0.10 per user/month for FingerprintJS, low for API limits).
- **Impact:** High (ensures fair pricing, supports reversals).
- **Complexity:** Moderate (SDK integration, rule updates).
- **Justification:** Polymarket's market depth deters bots without reversals, but ATHL's pricing curve and purchase-only phase require aggressive detection to prevent exploitation, critical for fairness and trust.

8. Regulatory Compliance Logs

Description: The system shall maintain encrypted logs of issuer authentications, Socure KYC results, and trade reversals in a secure database, accessible via RBAC, to support regulatory audits and compliance with GDPR/CCPA.

Threats Addressed: Regulatory exposure (securities violations, fraud accusations).

Implementation Details:

- Store issuer OAuth verifications, Socure KYC results, and reversal details (criteria, reasons) in AWS S3 (encrypted at rest/transit).
- Restrict log access with RBAC, requiring MFA for admins.
- Implement a dashboard for audit queries, exporting logs in CSV/JSON within 5 minutes.

- Retain logs for 7 years (per GDPR/CCPA), with automated deletion thereafter.
- Host logging logic on a cloud server (e.g., AWS Lambda).

Metrics:

- Log completeness: 100% of actions recorded.
- Audit response time: <24 hours for regulatory requests.
- Access violations: <0.1% of attempts.

Priority Rationale:

- **Cost:** Low (~\$0.01 per 1,000 log entries).
- **Impact:** High (avoids regulatory penalties).
- **Complexity:** Low (standard logging).
- **Justification:** Polymarket's blockchain logs are public but lack centralized audit needs, while ATHL's off-chain model and reversals require detailed logs to prove compliance, critical for speculative tokens.

9. Secure Issuer Publicity Toggle

Description: The system shall secure issuer publicity toggles for ATHLScan trades/holdings, requiring MFA for changes, logging actions, and notifying issuers within 5 minutes to prevent unauthorized disclosures.

Threats Addressed: Issuer fraud (unauthorized publicity), deanonymization.

Detection Criteria:

- Publicity toggle changes without MFA.
- Unauthorized toggle attempts (e.g., >3 failed MFA in 5 minutes).

Implementation Details:

- Require MFA (authenticator apps, email OTPs) for toggle changes in issuer account settings.
- Log toggle actions (timestamp, issuer ID, setting) in AWS RDS (encrypted), accessible via RBAC.

- Notify issuers via email (AWS SES) within 5 minutes of changes, with a revert link.
- Validate toggle requests server-side to prevent API tampering.
- Update ATHLSan UI to reflect publicity settings (e.g., “@ShaquilleONeal” vs. anon code).

Priority Rationale:

- **Cost:** Low (~\$0.0001 per OTP, \$0.01 per 1,000 logs).
- **Impact:** High (protects issuer privacy).
- **Complexity:** Low (MFA, UI updates).
- **Justification:** Polymarket’s public blockchain trades lack toggle mechanisms, but ATHL’s issuer publicity requires secure controls to prevent accidental or malicious exposure, aligning with privacy goals.

Integration with Existing Requirements

These additions complement your existing security requirements (e.g., Purchase Anomaly Detection, CAPTCHA, Social Media Monitoring, RBAC) by addressing Polymarket-specific gaps:

- **Social Verification (1, 2):** Enhances your Non-X/Grok-Based Social Media Monitoring by adding issuer authenticity checks, critical for social-driven manipulation absent in Polymarket.
- **Issuer MFA (3):** Extends your MFA requirement to issuers, strengthening account security beyond Polymarket’s wallet-based model.
- **Reversal Framework (4):** Builds on your Monitor Trading Patterns by formalizing reversal transparency, necessary for ATHL’s unique policy.
- **Cryptographic Signing (5):** Complements your ATHLSan Database Security (encryption, audits) to ensure off-chain integrity, unlike Polymarket’s blockchain.
- **ATHLSan Anonymization (6):** Integrates with your User Anonymity and Multiple Anonymous Codes, adding randomization and anti-scraping to protect privacy, surpassing Polymarket’s wallet pseudonymity.
- **Bot Detection (7):** Enhances your Behavioral Biometrics and Randomized Delays with tighter thresholds, addressing ATHL’s pricing curve risks.

- **Compliance Logs (8):** Aligns with your Compliance and User Support (GDPR/CCPA) for regulatory protection.
- **Publicity Toggle (9):** Supports your Issuer-Only Publicity feature with secure controls, unique to ATHL.

Additional Requirements To Do

- 1) AI chatbot pop up/agenting - notifications of what might pop/grow. - - real time alerting,
- 2) predictive analytics,
- 3) persona surfacing and matching for retargeting
 - a) PBTA - persona based targeting advertising - make this a requirement - a real time matching of issuer and fan to brand - for sponsorship opportunities. login and filter brands - taxonomy of brands - hitting audience profiles. real time matching of advertisers to opportunities. make this rough draft for now. It is a state model, captures all data. based on behavior no demographics. will patent this. real time predictive.
- 4)

KPIs

Dashboard Suite (Looker, Tableau, PowerBI, etc.)

Dashboard	Primary Audience	Key Visuals
Executive Overview	Founders, Investors	KPI scorecards, funnel waterfall, MRR from fees, top 5 liquidity tokens.
Onboarding & Verification	Growth & Security	Step-drop chart; heat-map of fails by social platform; median verification time by cohort.
Trading Activity	Product, Operations	Real-time token ticker; volume/price candlesticks; whale-watch (wallets > 2 000 tokens).

Referral & Rewards	Marketing	Referrals trend, LTV of referred users, reward payouts burn-down.
Issuer Performance	Issuers themselves , Customer Success, Account Management	Gauge: % of target raise; price over time; leaderboard position; follower velocity.
Security & Risk	Trust & Safety	Event timeline, risk score distribution, triggered pauses.
Support Ops	CX	Ticket backlog, SLA compliance gauge, word-cloud of ticket subjects.

<https://docs.google.com/spreadsheets/d/1KG21oY920eoL9bRtLfmTXA-a7GeYfHTs2k0YnAG1vw/edit?usp=sharing>